

SOLIDWORKS API (.NET VBA C#) — Examples (31/50):

- Get_Number_of_Lines_Flat-pattern_Drawing-
- Get_Preselected_Object_Example_VB

SOLIDWORKS API Documentation (converted from CHM)

Get the Number of Lines in Flat-pattern Drawing View's Boundary-box Sketch (VB.NET)

This example shows how to get the number of lines in a flat-pattern drawing view's boundary-box sketch.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\SMGussetAPI.SLDPRT.
' 2. Create a new drawing document.
' 3. Select SMGussetAPI.SLDPRT in the View
'    Palette's dropdown list box.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Examine the Immediate window and the drawing.
' 2. If necessary, drag the drawing onto the drawing sheet
'    and zoom in on the drawing view.
'
' NOTE: Because the part is used elsewhere, do not save
' changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swDrawing As DrawingDoc
        Dim swView As View
        Dim swSheet As Sheet
        Dim swDisplayData As DisplayData
        Dim sheetProperties As Object
        Dim sheetScale As Double
        Dim paperSize As swDwgPaperSizes_e
        Dim width As Double
        Dim height As Double
        Dim numViews As Long
        Dim viewName As Object
        Dim viewNames As Object
        Dim viewPaletteName As String = ""
        Dim drawingViewName As String = ""
        Dim status As Boolean

        swModel = swApp.ActiveDoc
        swDrawing = swModel

        ' Get current sheet
        swSheet = swDrawing.GetCurrentSheet
        sheetProperties = swSheet.GetProperties
        sheetScale = sheetProperties(2) / sheetProperties(3)
        paperSize = swSheet.GetSize(width, height)

        ' Get number of views on View Palette
        numViews = 0
        viewNames = swDrawing.GetDrawingPaletteViewNames

        ' Iterate through views on View Palette
        ' When view name equals "Flat pattern", drop
        ' that view in drawing
        If Not (IsNothing(viewNames)) Then
            numViews = (UBound(viewNames) - LBound(viewNames))
            For Each viewName In viewNames
                viewPaletteName = viewName
                If (viewPaletteName = "Flat pattern") Then
                    Debug.Print("Dropping View Palette view named: " & viewPaletteName)
                    swView = swDrawing.DropDrawingViewFromPalette2(viewPaletteName, 0.0#, 0.0#, 0.0#)
                    drawingViewName = swView.GetName2
                    Debug.Print("Dropped View Palette view into drawing view named: " & drawingViewName)
                    Exit For
                End If
            Next viewName
        End If

        ' Activate view and get number of lines in
        ' its boundary box sketch
        status = swDrawing.ActivateView(drawingViewName)
        swView = swDrawing.ActiveDrawingView

        ' Make sure the bounding box sketch is visible
        status = swModel.Extension.SelectByID2("Bounding-Box2@SMGussetAPI-1@DrawingView1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        swModel.UnblankSketch()

        swDisplayData = swView.GetSMBoundingBoxDisplayData2
        Debug.Print("Number of lines in boundary box of flat-pattern drawing view: " & swDisplayData.GetLineCount)

    End Sub

```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Get Number of Polylines in Shaded Mode Drawing View Example (VBA)

This example shows how to get the number of polylines in a drawing view displayed in Shaded and Wireframe modes.

```
'-----
' Preconditions:
' 1. Open public_documents\introsw\bolt-assembly.slddrw.
' 2. Change Drawing View4 to Shaded mode (click
'   Drawing View4 in the drawing, click Shaded
'   in Display Style in the PropertyManager page, and
'   click OK.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the number of polylines in Shaded mode.
' 2. Switches mode to Wireframe and gets number of polylines
'   in Wireframe mode.
' 3. Switches mode back to Shaded.
' 4. Examine the Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save
' changes
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swDrawDoc As SldWorks.DrawingDoc
Dim swView As SldWorks.View
Dim retval As Long
Dim boolstatus As Boolean
Dim nCountShaded As Long
Dim nCount_WireFrame As Long
Dim retval2 As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swDrawDoc = swApp.ActiveDoc

    swDrawDoc.ClearSelection2 True

    ' Select Drawing View4
    boolstatus = swDrawDoc.Extension.SelectByID2("Drawing View4", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)
    Set swView = swDrawDoc.SelectionManager.GetSelectedObject6(1, -1)

    ' Get display mode for Drawing View4
    Set swView = swDrawDoc.ActiveDrawingView
    retval = swView.GetDisplayMode2
    Debug.Print "Drawing view in what display mode"
    Debug.Print "    swDisplayMode_e: swWIREFRAME = 0, swHIDDEN_GREYED = 1, swHIDDEN = 2, swSHADED = 3: " & retval

    ' Get number of polylines in drawing view in Shaded mode
    swView.GetPolyLineCount5 swCrossHatchFilter_e.swCrossHatchExclude, nCountShaded
    Debug.Print "Number of polylines in drawing view in Shaded mode = " & nCountShaded

    ' Switch to Wireframe mode because you can get number of polylines in Wireframe mode
    retval2 = swView.SetDisplayMode3(False, swDisplayMode_e.swWIREFRAME, False, False)
    Debug.Print "Display mode changed to Wireframe: " & retval2

    ' Get number of polylines in Wireframe mode
    swView.GetPolyLineCount5 swCrossHatchFilter_e.swCrossHatchExclude, nCount_WireFrame
    Debug.Print "Number of polylines in WireFrame = " & nCount_WireFrame

    ' Switch back to Shaded mode
    retval2 = swView.SetDisplayMode3(False, swDisplayMode_e.swSHADED, False, False)
    Debug.Print "Display mode changed back to Shaded: " & retval2

    swDrawDoc.ClearSelection2 True

End Sub
```

Get Object ID of GTol Annotation Example (C#)

This example shows how to get the object ID of a GTol annotation.

```
//-----
// Preconditions:
// 1. Verify that the drawing to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified drawing.
// 2. Selects the GTol annotation.
// 3. Gets the type of annotation selected.
// 4. Gets the selected GTol annotation's object ID.
// 5. Examine the Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GetObjectIDCSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDrawing = default(DrawingDoc);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            GTol swGTol = default(GTol);
            Annotation swGTolAnnotation = default(Annotation);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            int id = 0;

            //Open the drawing
            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\gtolwitnessline.slddrw", 3, 0, "", ref errors, ref warnings);
            swDrawing = (DrawingDoc)swModel;

            //Select GTol
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("DetailItem40@Drawing View3", "GTOL", 0.44609485235144, 0.381203477032446, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swGTol = (GTol)swSelectionMgr.GetSelectedObject6(1, -1);

            //Print type of annotation and its object ID
            swGTolAnnotation = (Annotation)swGTol.GetAnnotation();
            Debug.Print("Annotation type: " + swGTolAnnotation.GetType());
            id = swModelDocExt.GetObjectId(swGTolAnnotation);
            Debug.Print("ID = " + id);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Object ID of GTol Annotation Example (VBA)

This example shows how to get the object ID of a GTol annotation.

```
'-----  
' Preconditions:  
' 1. Verify that the drawing to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified drawing.  
' 2. Selects the GTol annotation.  
' 3. Gets the type of annotation selected.  
' 4. Gets the selected GTol annotation's object ID.  
' 5. Examine the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swDrawing As SldWorks.DrawingDoc  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelectionMgr As SldWorks.SelectionMgr  
Dim swGTol As SldWorks.GTol  
Dim swGTolAnnotation As SldWorks.Annotation  
Dim status As Boolean  
Dim errors As Long, warnings As Long  
Dim id As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
  
    'Open the drawing  
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\gtolwitnessline.slddrw", 3, 0, "", errors,  
warnings)  
    Set swDrawing = swModel  
  
    'Select GTol  
    Set swModelDocExt = swModel.Extension  
    status = swModelDocExt.SelectByID2("DetailItem40@Drawing View3", "GTOL", 0.44609485235144, 0.381203477032446, 0, False, 0, Nothing, 0)  
    Set swSelectionMgr = swModel.SelectionManager  
    Set swGTol = swSelectionMgr.GetSelectedObject6(1, -1)  
  
    'Print type of annotation and its object ID  
    Set swGTolAnnotation = swGTol.GetAnnotation  
    Debug.Print "Annotation type: " & swGTolAnnotation.GetType  
    id = swModelDocExt.GetObjectId(swGTolAnnotation)  
    Debug.Print "ID = " & id  
  
End Sub
```

Get Object ID of GTol Annotation Example (VB.NET)

This example shows how to get the object ID of a GTol annotation.

```
'-----
' Preconditions:
' 1. Verify that the drawing to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing.
' 2. Selects the GTol annotation.
' 3. Gets the type of annotation selected.
' 4. Gets the selected GTol annotation's object ID.
' 5. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swDrawing As DrawingDoc
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swGTol As GTol
        Dim swGTolAnnotation As Annotation
        Dim status As Boolean
        Dim errors As Integer, warnings As Integer
        Dim id As Integer

        'Open the drawing
        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\gtolwitnessline.sldrw", 3, 0, "", errors, warnings)
        swDrawing = swModel

        'Select GTol
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("DetailItem40@Drawing View3", "GTOL", 0.44609485235144, 0.381203477032446, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swGTol = swSelectionMgr.GetSelectedObject6(1, -1)

        'Print type of annotation and its object ID
        swGTolAnnotation = swGTol.GetAnnotation
        Debug.Print("Annotation type: " & swGTolAnnotation.GetType)
        id = swModelDocExt.GetObjectId(swGTolAnnotation)
        Debug.Print("ID = " & id)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Get Object's Persistent Reference ID Example (C++)

This example shows how to:

- get the selected object's persistent reference ID using IModelDocExtension::GetPersistReference3.
- use the ISafeArrayUtility to parse the Variant SafeArray returned by IModelDocExtension::GetPersistReference3.
- get the size of the persistent reference ID for the selected object.

```
//-----
// Preconditions:
// 1. Start Microsoft Visual Studio 2019 or later.
// a. Click File > New > Project > (C++ Windows) Console App.
// b. Click Next.
// c. Type the name of your project in Project Name.
// d. Click Create.
// e. Right-click project, select Add, New Item, Select ATL > ATL Support, Click Add.
// f. Click Project > projectname Properties > Configuration Properties >
// C/C++ and type the path to sldworks.tlb and swconst.tlb,
// typically C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS,
// in Additional Include Directories.
// g. Click Apply or OK.
// 2. Replace the code in the code window with This code.
// 3. Click Debug > Start Debugging.
//
// Postconditions:
// 1. Gets each byte value in a Variant SafeArray of byte values
// of the persistent reference ID.
// 2. Displays a message box showing the size of of the persistent
// reference ID assigned to the selected object.
// 3. Click OK to close the message box.
//-----
//This code

#include "framework.h"
#include <string>

#import "sldworks.tlb" raw_interfaces_only, raw_native_types, no_namespace, named_guids //SOLIDWORKS type library
#import "swconst.tlb" no_namespace, raw_native_types, no_namespace, named_guids //SOLIDWORKS constants type library

template <typename T>
std::string number_to_string(T number)
{
    return dynamic_cast<std::ostringstream *> (&(std::stringstream() << number))->str();
}

int _tmain(int argc, TCHAR* argv[])
{
    //Initialize COM
    //Do this before using ATL smart pointers so that
    //COM is available
    CoInitialize(NULL);

    //Use a block so that the smart pointers are destructed when
    //scope of this block is exited
    {
        //Use ATL smart pointers
        CComPtr<ISldWorks> swApp;
        if(swApp.CoCreateInstance(L"SldWorks.Application", NULL, CLSCTX_LOCAL_SERVER) != S_OK){
            return(0);
        }

        CComPtr<IModelDoc2> swDoc;
        swApp->get_IActiveDoc2(&swDoc);

        CComPtr<IModelDocExtension> swDocExt;
        swDoc->get_Extension(&swDocExt);

        CComPtr<ISelectionMgr> swSelMgr;
        swDoc->get_ISelectionManager(&swSelMgr);

        CComPtr<IDispatch> disp;
        swSelMgr->GetSelectedObject6(1, -1, &disp);

        //Get the persistent reference ID for the selected object
        _variant_t varPersist;
        ::VariantInit(&varPersist);
        swDocExt->GetPersistReference3(disp, &varPersist);

        CComPtr<IDispatch> pDispatchSafeArray = NULL;
        CComPtr<ISafeArrayUtility> swSafeArray = NULL;
        HRESULT hres;
        hres = swApp->GetSafeArrayUtility(&pDispatchSafeArray);
        hres = pDispatchSafeArray.QueryInterface<ISafeArrayUtility>(&swSafeArray);

        long sizePersistIDs;
        swDocExt->GetPersistReferenceCount3(disp, &sizePersistIDs);

        for(int i = 0; i < sizePersistIDs; i++){
            byte byteValue;
            //Get each byte value in the Variant SafeArray of byte values
            swSafeArray->GetByte(varPersist, i, &byteValue);
            //TODO: Do something with each persistent reference ID
        }

        _bstr_t s;
        s = (long)sizePersistIDs;
        CComBSTR msg;
        msg = (OLESTR("Size of persistent reference ID: "));
        msg.AppendBSTR(s);
        long res;
        //Pop up a message box with the size of persistent
        //reference ID for the selected object
        swApp->SendMessageToUser2(msg, 0, 0, &res);
    }

    //ATL smart pointers are destructed so that all COM objects
    //held on to are released
    //Shut down COM because you no longer need it
}
```

```
CoUninitialize();  
return(0);
```

```
}
```

Get Object's Persistent Reference ID Example (VBA)

This example shows how to get an object's persistent reference IDs.

```
'-----
' Preconditions:
' 1. Open a part that contains an Extrude1 feature.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the persistent reference ID of the Extrude1 feature
'    after after releasing its object, after rebuilding the part,
'    after closing and opening the part document.
' 2. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim docext As SldWorks.ModelDocExtension
Dim pathname As String
Dim filetype As Long
Dim errors As Long
Dim warnings As Long
Dim options As Long
Dim obj As Object
Dim reference As Variant
Dim index As Integer
Dim boolstatus As Boolean
Dim errorCode As Long

Sub main()

    Set swApp = Application.SldWorks

    'Get the document
    Set Part = swApp.ActiveDoc
    Set docext = Part.Extension

    'Get the object
    boolstatus = Part.Extension.SelectByID2("Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)
    Set obj = Part.SelectionManager.GetSelectedObject5(1)

    'Get the object's reference id
    reference = docext.GetPersistReference3(obj)

    'After releasing the object with the part remaining available
    Set obj = Nothing
    Set obj = docext.GetObjectByPersistReference3(reference, errorCode)
    Debug.Print "Get object by persistent reference ID after releasing object: " & errorCode

    'After rebuilding the part
    Set obj = Nothing
    Call Part.Rebuild(swForceRebuildAll)
    Set obj = docext.GetObjectByPersistReference3(reference, errorCode)
    Debug.Print "Get object by persistent reference ID after rebuilding part: " & errorCode

    'After closing and opening the part
    Set obj = Nothing
    pathname = Part.GetPathName
    swApp.CloseDoc pathname
    Set Part = Nothing
    Set docext = Nothing
    Set Part = swApp.GetOpenDocumentByName(pathname)
    Set Part = swApp.OpenDoc6(pathname, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)
    Set docext = Part.Extension
    Set obj = docext.GetObjectByPersistReference3(reference, errorCode)
    Debug.Print "Get object by persistent reference ID after closing and opening part: " & errorCode

End Sub
```

Get Objects in Selection Set Example (C#)

This example shows how to get the objects in a selection set.

```
//-----
// Preconditions:
// 1. Verify that the assembly to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the assembly.
// 2. Creates two selection sets.
// 3. Traverses the nodes in the FeatureManager design tree. If the
//    name of the node is Selection-Set1(3), then:
//    a. Gets the type of feature for this node.
//    b. Gets the Selection Sets folder.
//    c. Gets the selection set by name.
//    d. Suspends the current selection list, starts a new selection list,
//    and adds the objects in the selection set to the new selection
//    list.
//    e. Gets the number of objects in the selection list and traverses
//    the selection list.
//    1. Gets the type of object in the selection list.
//    2. Casts the object to a component.
//    3. Gets the name of the component.
//    f. Reinstates the suspended selection list.
// 4. Examines the Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            FeatureManager swFeatureMgr = default(FeatureManager);
            TreeControlItem rootNode = default(TreeControlItem);
            ModelDocExtension swModelDocExtension = default(ModelDocExtension);
            SelectionSet swSelectionSet1 = default(SelectionSet);
            SelectionSet swSelectionSet2 = default(SelectionSet);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\wrench.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
            rrors, ref warnings);
            Debug.Print("File = " + swModel.GetPathName());
            swModelDocExtension = (ModelDocExtension)swModel.Extension;

            //Create a selection set
            status = swModelDocExtension.SelectByID2("arm1-1@wrench", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExtension.SelectByID2("arm2-1@wrench", "COMPONENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExtension.SelectByID2("clamp2-1@wrench", "COMPONENT", 0, 0, 0, true, 0, null, 0);
            swSelectionSet1 = (SelectionSet)swModelDocExtension.SaveSelection(out errors);
            swModel.ClearSelection2(true);

            //Create another selection set
            status = swModelDocExtension.SelectByID2("centerlink-1@wrench", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExtension.SelectByID2("screw-1@wrench", "COMPONENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExtension.SelectByID2("screw-1@wrench", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swSelectionSet2 = (SelectionSet)swModelDocExtension.SaveSelection(out errors);
            swModel.ClearSelection2(true);

            swFeatureMgr = (FeatureManager)swModel.FeatureManager;
            rootNode = (TreeControlItem)swFeatureMgr.GetFeatureTreeRootItem();
            if ((rootNode != null))
            {
                traverse_node(rootNode, swFeatureMgr, swModel);
            }
        }

        private void traverse_node(TreeControlItem node, FeatureManager swFeatureMgr, ModelDoc2 swModel)
        {
            TreeControlItem childNode = default(TreeControlItem);
            Feature featureNode = default(Feature);
            int nodeObjectType = 0;
            object nodeObject = null;

            nodeObjectType = node.ObjectType;
            nodeObject = (object)node.Object;
            switch (nodeObjectType)
            {
                case (int)swTreeControlItemType_e.swFeatureManagerItem_Feature:
                    if ((nodeObject != null))
                    {
                        featureNode = (Feature)nodeObject;
                        if (featureNode.Name == "Selection-Set1(3)")
                        {
                            Feature swSelectionSetFeature = default(Feature);
                            SelectionSetFolder swSelectionSetFolderFeature = default(SelectionSetFolder);
                            SelectionSet swSelectionSet = default(SelectionSet);
                            Component2 swComponent = default(Component2);
                            SelectionMgr swSelectionMgr = default(SelectionMgr);
                            int nbrSelections = 0;
                            int i = 0;
                            object selObj = null;

                            swSelectionSetFeature = (Feature)node.Object;
                            Debug.Print("Feature name: " + swSelectionSetFeature.Name);
                        }
                    }
            }
        }
    }
}
```

```

        Debug.Print(" Feature type: " + swSelectionSetFeature.GetTypeName2());
        swSelectionSetFolderFeature = (SelectionSetFolder)swFeatureMgr.GetSelectionSetFolder();
        swSelectionSet = (SelectionSet)swSelectionSetFolderFeature.GetSelectionSetByName(swSelectionSetFeature.Name);
        Debug.Print("Selection set name: " + swSelectionSet.GetName());

        swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
        swSelectionMgr.SuspendSelectionList();

        swSelectionSet.Select();

        nbrSelections = swSelectionMgr.GetSelectedObjectCount2(-1);
        Debug.Print(" Number of objects selected in this selection set: " + nbrSelections);
        for (i = 1; i <= nbrSelections; i++)
        {
            //Get each object in the new selection list
            selObj = (object)swSelectionMgr.GetSelectedObject6(i, -1);
            Debug.Print(" Object[" + i + "] in the selection list is of swSelectType_e = " + swSelectionMgr.GetSelectedObjectType3(
i, -1) + "");

            //Cast the object
            swComponent = (Component2)selObj;
            Debug.Print(" Name of component: " + swComponent.Name2);
        }
        swSelectionMgr.ResumeSelectionList();
    }
    }
    break;
default:
    break;
}
childNode = (TreeControlItem)node.GetFirstChild();
while ((childNode != null))
{
    traverse_node(childNode, swFeatureMgr, swModel);
    childNode = (TreeControlItem)childNode.GetNext();
}
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Get Objects in Selection Set Example (VBA)

This example shows how to get the objects in a selection set.

```

'-----
' Preconditions:
' 1. Verify that the assembly to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the assembly.
' 2. Creates two selection sets.
' 3. Traverses the nodes in the FeatureManager design tree. If the
'    name of the node is Selection-Set1(3), then:
'    a. Gets the type of feature for this node.
'    b. Gets the Selection Sets folder.
'    c. Gets the selection set by name.
'    d. Suspends the current selection list, starts a new selection list,
'       and adds the objects in the selection set to the new selection
'       list.
'    e. Gets the number of objects in the selection list and traverses
'       the selection list.
'       1. Gets the type of object in the selection list.
'       2. Casts the object to a component.
'       3. Gets the name of the component.
'    f. Reinstates the suspended selection list.
' 4. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swFeatureMgr As SldWorks.FeatureManager
    Dim rootNode As SldWorks.TreeControlItem
    Dim swModelDocExtension As SldWorks.ModelDocExtension
    Dim swSelectionSet1 As SldWorks.SelectionSet
    Dim swSelectionSet2 As SldWorks.SelectionSet
    Dim status As Boolean
    Dim errors As Long
    Dim warnings As Long
    Dim fileName As String

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\wrench.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Debug.Print "File = " & swModel.GetPathName
    Set swModelDocExtension = swModel.Extension

    'Create a selection set
    status = swModelDocExtension.SelectByID2("arm1-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    status = swModelDocExtension.SelectByID2("arm2-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExtension.SelectByID2("clamp2-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
    Set swSelectionSet1 = swModelDocExtension.SaveSelection(errors)
    swModel.ClearSelection2 True

    'Create another selection set
    status = swModelDocExtension.SelectByID2("centerlink-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    status = swModelDocExtension.SelectByID2("screw-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExtension.SelectByID2("screw-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionSet2 = swModelDocExtension.SaveSelection(errors)
    swModel.ClearSelection2 True

    Set swFeatureMgr = swModel.FeatureManager
    Set rootNode = swFeatureMgr.GetFeatureTreeRootItem()
    If Not rootNode Is Nothing Then
        traverse_node rootNode, swFeatureMgr, swModel
    End If
End Sub

Private Sub traverse_node(node As SldWorks.TreeControlItem, swFeatureMgr As SldWorks.FeatureManager, swModel As SldWorks.ModelDoc2)

    Dim childNode As SldWorks.TreeControlItem
    Dim featureNode As SldWorks.Feature
    Dim nodeObjectType As Long
    Dim nodeObject As Object

    nodeObjectType = node.ObjectType
    Set nodeObject = node.Object

    Select Case nodeObjectType
        Case swTreeControlItemType_e.swFeatureManagerItem_Feature:
            If Not nodeObject Is Nothing Then
                Set featureNode = nodeObject
                If featureNode.Name = "Selection-Set1(3)" Then

                    Dim swSelectionSetFeature As SldWorks.Feature
                    Dim swSelectionSetFolderFeature As SldWorks.SelectionSetFolder
                    Dim swSelectionSet As SldWorks.SelectionSet
                    Dim swComponent As SldWorks.Component2
                    Dim swSelectionMgr As SldWorks.SelectionMgr
                    Dim nbrSelections As Long

```

```

Dim i As Long
Dim selObj As Object

Set swSelectionSetFeature = node.Object
Debug.Print "Feature name: " & swSelectionSetFeature.Name
Debug.Print "   Feature type: " & swSelectionSetFeature.GetType
Set swSelectionSetFolderFeature = swFeatureMgr.GetSelectionSetFolder

Set swSelectionSet = swSelectionSetFolderFeature.GetSelectionSetByName(swSelectionSetFeature.Name)
Debug.Print "Selection set name: " & swSelectionSet.GetName

Set swSelectionMgr = swModel.SelectionManager
swSelectionMgr.SuspendSelectionList

swSelectionSet.Select

nbrSelections = swSelectionMgr.GetSelectedObjectCount2(-1)
Debug.Print "   Number of objects selected in this selection set: " & nbrSelections

For i = 1 To nbrSelections
    'Get each object in the new selection list
    Set selObj = swSelectionMgr.GetSelectedObject6(i, -1)
    Debug.Print "       Object[" & i & "] in the selection list is of swSelectType_e = " & swSelectionMgr.GetSelectedObjectType3(i, -1) &
    'Cast the object
    Set swComponent = selObj
    Debug.Print "           Name of component: " & swComponent.Name2
Next

swSelectionMgr.ResumeSelectionList
End If
End If

End Select

Set childNode = node.GetFirstChild()
While Not childNode Is Nothing
    traverse_node childNode, swFeatureMgr, swModel
    Set childNode = childNode.GetNext
Wend

End Sub

```

Get Objects in Selection Set Example (VB.NET)

This example shows how to get the objects in a selection set.

```

'-----
' Preconditions:
' 1. Verify that the assembly to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the assembly.
' 2. Creates two selection sets.
' 3. Traverses the nodes in the FeatureManager design tree. If the
'    name of the node is Selection-Set1(3), then:
'    a. Gets the type of feature for this node.
'    b. Gets the Selection Sets folder.
'    c. Gets the selection set by name.
'    d. Suspends the current selection list, starts a new selection list,
'       and adds the objects in the selection set to the new selection
'       list.
'    e. Gets the number of objects in the selection list and traverses
'       the selection list.
'       1. Gets the type of object in the selection list.
'       2. Casts the object to a component.
'       3. Gets the name of the component.
'    f. Reinstates the suspended selection list.
' 4. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swFeatureMgr As FeatureManager
        Dim rootNode As TreeControlItem
        Dim swModelDocExtension As ModelDocExtension
        Dim swSelectionSet1 As SelectionSet
        Dim swSelectionSet2 As SelectionSet
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\wrench.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        Debug.Print("File = " & swModel.GetPathName)
        swModelDocExtension = swModel.Extension

        'Create a selection set
        status = swModelDocExtension.SelectByID2("arm1-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        status = swModelDocExtension.SelectByID2("arm2-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExtension.SelectByID2("clamp2-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
        swSelectionSet1 = swModelDocExtension.SaveSelection(errors)
        swModel.ClearSelection2(True)

        'Create another selection set
        status = swModelDocExtension.SelectByID2("centerlink-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        status = swModelDocExtension.SelectByID2("screw-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExtension.SelectByID2("screw-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionSet2 = swModelDocExtension.SaveSelection(errors)
        swModel.ClearSelection2(True)

        swFeatureMgr = swModel.FeatureManager
        rootNode = swFeatureMgr.GetFeatureTreeRootItem()
        If Not rootNode Is Nothing Then
            traverse_node(rootNode, swFeatureMgr, swModel)
        End If

    End Sub

    Private Sub traverse_node(ByVal node As TreeControlItem, ByVal swFeatureMgr As FeatureManager, ByVal swModel As ModelDoc2)
        Dim childNode As TreeControlItem
        Dim featureNode As Feature
        Dim nodeObjectType As Integer
        Dim nodeObject As Object

        nodeObjectType = node.ObjectType
        nodeObject = node.Object
        Select Case nodeObjectType
            Case swTreeControlItemType_e.swFeatureManagerItem_Feature
                If Not nodeObject Is Nothing Then
                    featureNode = nodeObject
                    If featureNode.Name = "Selection-Set1(3)" Then

                        Dim swSelectionSetFeature As Feature
                        Dim swSelectionSetFolderFeature As SelectionSetFolder
                        Dim swSelectionSet As SelectionSet
                        Dim swComponent As Component2
                        Dim swSelectionMgr As SelectionMgr
                        Dim nbrSelections As Integer
                        Dim i As Integer
                        Dim selObj As Object

                        swSelectionSetFeature = node.Object
                    End If
                End If
            End Select
        End Sub
    
```

```

Debug.Print("Feature name: " & swSelectionSetFeature.Name)
Debug.Print(" Feature type: " & swSelectionSetFeature.GetTypeName2)
swSelectionSetFolderFeature = swFeatureMgr.GetSelectionSetFolder
swSelectionSet = swSelectionSetFolderFeature.GetSelectionSetByName(swSelectionSetFeature.Name)
Debug.Print("Selection set name: " & swSelectionSet.GetName)

swSelectionMgr = swModel.SelectionManager
swSelectionMgr.SuspendSelectionList()

swSelectionSet.Select()

nbrSelections = swSelectionMgr.GetSelectedObjectCount2(-1)
Debug.Print(" Number of objects selected in this selection set: " & nbrSelections)
For i = 1 To nbrSelections
    'Get each object in the new selection list
    selObj = swSelectionMgr.GetSelectedObject6(i, -1)
    Debug.Print(" Object[" & i & "] in the selection list is of swSelectType_e = " & swSelectionMgr.GetSelectedObjectType3(i, -
1) & "")
    'Cast the object
    swComponent = selObj
    Debug.Print(" Name of component: " & swComponent.Name2)
Next

swSelectionMgr.ResumeSelectionList()
End If
End If

End Select

childNode = node.GetFirstChild()
While Not childNode Is Nothing
    traverse_node(childNode, swFeatureMgr, swModel)
    childNode = childNode.GetNext
End While

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Get Offset Surface Data Example (C#)

This example shows how to get data for an offset surface.

```
//-----
// Preconditions:
// 1. Open an assembly document that contains a component that
//    has a surface offset feature.
// 2. Select the component's surface offset feature.
// 3. Open the Immediate window.
//
// Postconditions: Inspect the Immediate window.
//-----

using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace AnalyzeOffsetSurface_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SelectData swSelData = default(SelectData);
            SurfaceOffsetFeatureData swOffset = default(SurfaceOffsetFeatureData);
            Feature swFeat = default(Feature);
            Entity swEnt = default(Entity);
            object[] vFace = null;
            int i = 0;
            bool bRet = false;
            Component2 comp = default(Component2);
            Component2 swCompFace = default(Component2);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelData = (SelectData)swSelMgr.CreateSelectData();
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swOffset = (SurfaceOffsetFeatureData)swFeat.GetDefinition();
            comp = (Component2)swSelMgr.GetSelectedObjectsComponent3(1, -1);

            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print("CompFeature = " + comp.Name2);
            Debug.Print(" " + swFeat.Name);
            Debug.Print("    Distance    = " + swOffset.Distance * 1000.0 + " mm");
            Debug.Print("    Flip        = " + swOffset.Flip);
            Debug.Print("    FacesCount  = " + swOffset.GetEntitiesCount());

            bRet = swOffset.AccessSelections(swModel, comp);

            swModel.ClearSelection2(true);

            vFace = (object[])swOffset.Entities;

            for (i = 0; i <= vFace.GetUpperBound(0); i++)
            {
                swEnt = (Entity)vFace[i];

                Debug.Print(" Entity selected = " + swEnt.Select4(true, null));

                swCompFace = (Component2)swEnt.GetComponent();
                Debug.Print("    Component face = " + swCompFace.Name2);
            }

            swOffset.ReleaseSelectionAccess();
        }

        public SldWorks swApp;
    }
}
```

Get Offset Surface Data Example (VBA)

This example shows how to get data for an offset surface.

```

'-----
' Preconditions:
' 1. Open an assembly document that contains a component that
'    has a surface offset feature.
' 2. Select the component's surface offset feature.
' 3. Open the Immediate window.
'
' Postconditions: Inspect the Immediate window.
'-----

```

Option Explicit

```

Sub main()

    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim swSelMgr             As SldWorks.SelectionMgr
    Dim swSelData            As SldWorks.SelectData
    Dim swOffset             As SldWorks.SurfaceOffsetFeatureData
    Dim swFeat              As SldWorks.Feature
    Dim swEnt                As SldWorks.Entity
    Dim vFace               As Variant
    Dim i                   As Long
    Dim bRet                As Boolean
    Dim comp                As SldWorks.Component2
    Dim swCompFace          As SldWorks.Component2

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swOffset = swFeat.GetDefinition
    Set comp = swSelMgr.GetSelectedObjectsComponent3(1, -1)

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print "CompFeature = " & comp.Name2
    Debug.Print " " & swFeat.Name
    Debug.Print "    Distance      = " & swOffset.Distance * 1000# & " mm"
    Debug.Print "    Flip          = " & swOffset.Flip
    Debug.Print "    FacesCount    = " & swOffset.GetEntitiesCount

    bRet = swOffset.AccessSelections(swModel, comp)
    swModel.ClearSelection2 True

    vFace = swOffset.Entities
    For i = 0 To UBound(vFace)
        Set swEnt = vFace(i)
        Debug.Print "    Entity selected = " & swEnt.Select4(True, Nothing)

        Set swCompFace = swEnt.GetComponent
        Debug.Print "    Component face = " & swCompFace.Name2
    Next i

    swOffset.ReleaseSelectionAccess
End Sub

```

Get Offset Surface Data Example (VB.NET)

This example shows how to get data for an offset surface.

```

'-----
' Preconditions:
' 1. Open an assembly document that contains a component that
'    has a surface offset feature.
' 2. Select the component's surface offset feature.
' 3. Open the Immediate window.
'-----
' Postconditions: Inspect the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swSelData As SelectData
        Dim swOffset As SurfaceOffsetFeatureData
        Dim swFeat As Feature
        Dim swEnt As Entity
        Dim vFace As Object
        Dim i As Long
        Dim bRet As Boolean
        Dim comp As Component2
        Dim swCompFace As Component2

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swSelData = swSelMgr.CreateSelectData
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swOffset = swFeat.GetDefinition
        comp = swSelMgr.GetSelectedObjectsComponent3(1, -1)

        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print("CompFeature = " & comp.Name2)
        Debug.Print(" " & swFeat.Name)
        Debug.Print("    Distance      = " & swOffset.Distance * 1000.0# & " mm")
        Debug.Print("    Flip          = " & swOffset.Flip)
        Debug.Print("    FacesCount    = " & swOffset.GetEntitiesCount)

        bRet = swOffset.AccessSelections(swModel, comp)

        swModel.ClearSelection2(True)

        vFace = swOffset.Entities

        For i = 0 To UBound(vFace)
            swEnt = vFace(i)
            Debug.Print "    Entity selected = " & swEnt.Select4(True, Nothing)
            swCompFace = swEnt.GetComponent
            Debug.Print("    Component face = " & swCompFace.Name2)
        Next i

        swOffset.ReleaseSelectionAccess()

    End Sub

    Public swApp As SldWorks

End Class

```

Get OLE Object Data Example (C#)

This example shows how to get OLE object data from a model document.

```
//-----
// Preconditions:
// 1. Open a model document.
// 2. Click Insert > Object.
// 3. Click Paintbrush Picture > OK.
// 4. Click File > Close.
// 5. Open the Immediate window.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetOLEObjectData_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        SelectionMgr swSelMgr;
        SwOLEObject swOLEObj;
        Boolean boolstatus;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            swOLEObj = (SwOLEObject)swSelMgr.GetSelectedObject6(1, 0);

            Debug.Print("OLE Object");
            Debug.Print("  Class ID: " + swOLEObj.Clsid);
            Debug.Print("  Is linked? " + swOLEObj.IsLinked);
            Debug.Print("  Filename: " + swOLEObj.FileName);
            Debug.Print("  Buffer size: " + swOLEObj.BufferSize);
            Debug.Print("  Viewing aspect (1=Content, 2=Thumbnail, 4=Icon, 8=DocPrint): " + swOLEObj.Aspect);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Get OLE Object Data Example (VBA)

This example shows how to get OLE object data from a model document.

```
'-----  
' Preconditions:  
' 1. Open a model document.  
' 2. Click Insert > Object.  
' 3. Click Paintbrush Picture > OK.  
' 4. Click File > Close.  
' 5. Open the Immediate window.  
'  
' Postconditions: Inspect the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
  
Dim swModel As SldWorks.ModelDoc2  
  
Dim swModelDocExt As SldWorks.ModelDocExtension  
  
Dim swSelMgr As SldWorks.SelectionMgr  
  
Dim swOLEObj As SldWorks.SwOLEObject  
  
Dim boolstatus As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
    Set swSelMgr = swModel.SelectionManager  
  
    Set swOLEObj = swSelMgr.GetSelectedObject6(1, 0)  
  
    Debug.Print "OLE Object"  
    Debug.Print "  Class ID: " & swOLEObj.Clsid  
    Debug.Print "  Is linked? " & swOLEObj.IsLinked  
    Debug.Print "    Filename: " & swOLEObj.FileName  
    Debug.Print "  Buffer size: " & swOLEObj.BufferSize  
    Debug.Print "  Viewing aspect (1=Content, 2=Thumbnail, 4=Icon, 8=DocPrint): " & swOLEObj.Aspect  
  
End Sub
```

Get OLE Object Data Example (VB.NET)

This example shows how to get OLE object data from a model document.

```
'-----
' Preconditions:
' 1. Open a model document.
' 2. Click Insert > Object.
' 3. Click Paintbrush Picture > OK.
' 4. Click File > Close.
' 5. Open the Immediate window.
'
' Postconditions: Inspect the Immediate window.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swSelMgr As SelectionMgr
    Dim swOLEObj As SwOLEObject
    Dim boolStatus As Boolean
```

```
    Sub main()
```

```
        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
```

```
        swOLEObj = swSelMgr.GetSelectedObject6(1, 0)
```

```
        Debug.Print("OLE Object")
        Debug.Print("  Class ID: " & swOLEObj.Clsid)
        Debug.Print("  Is linked? " & swOLEObj.IsLinked)
        Debug.Print("  Filename: " & swOLEObj.FileName)
        Debug.Print("  Buffer size: " & swOLEObj.BufferSize)
        Debug.Print("  Viewing aspect (1=Content, 2=Thumbnail, 4=Icon, 8=DocPrint): " & swOLEObj.Aspect)
```

```
    End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Get or Set the Backup Directory Example (VBA)

This example shows how to set and get the backup directory.

```
'-----  
' Preconditions:  
' 1. Verify that c:\temp exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Sets c:\temp as the backup directory.  
' 2. Gets the name of the backup directory.  
' 3. Examine the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim backupDirectory As String  
Dim status As Boolean  
  
Sub main()  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    backupDirectory = "c:\temp"  
    status = swApp.SetUserPreferenceStringValue(swUserPreferenceStringValue_e.swBackupDirectory, backupDirectory)  
    backupDirectory = swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swBackupDirectory)  
    Debug.Print "Backup directory: " & backupDirectory  
  
End Sub
```

Get or Set Whether Edits Update Design Table Example (VBA)

This example shows how to get or set whether edits to a design table update the design table.

```
'-----
' Preconditions:
' 1. Open a part document that contains a design table.
' 2. Verify that the part document contains a design
'    table by expanding Tables in the ConfigurationManager.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets whether the design table is updatable before
'    running the macro.
' 2. Sets the design table to not updatable if it is
'    updatable or vice versa.
' 3. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDesignTable As SldWorks.DesignTable
Dim boolstatus As Boolean
Dim laststate As Variant

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    Set swDesignTable = swModel.GetDesignTable()

    ' Allow changes to the characteristics of the
    ' design table
    swDesignTable.EditFeature

    ' Get whether design table is updatable
    laststate = swDesignTable.Updatable
    Debug.Print "Design table updatable: " & CStr(laststate)

    ' If design table is updatable, then set it
    ' to not updatable or vice versa
    swDesignTable.Updatable = Not laststate

    ' Update this characteristic of the design table
    boolstatus = swDesignTable.UpdateFeature()

    ' Get whether design table is updatable
    laststate = swDesignTable.Updatable
    Debug.Print "Design table updatable: " & CStr(laststate)

End Sub
```

Get Original Body from Pattern Body (C#)

This example shows how to get the original body from a pattern body.

```
//-----
// Preconditions:
// 1. Open a part document that contains a pattern of solid bodies.
// 2. Select a pattern body from the Solid Bodies folder.
//
// Postconditions:
// 1. The original body from which the pattern was generated is hidden from view.
// 2. Click F5 to display the original body.
// -----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System.Windows.Forms;

namespace GetOriginalBodyFromPattern_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swDoc;
        SelectionMgr swSm;
        int selType;
        Body2 swBody;
        Body2 swOriBody;

        MathTransform swMathTrans;

        public void Main()
        {
            swDoc = (ModelDoc2)swApp.ActiveDoc;
            swSm = (SelectionMgr)swDoc.SelectionManager;

            selType = swSm.GetSelectedObjectType3(1, -1);
            swBody = (Body2)swSm.GetSelectedObject6(1, -1);
            Debug.Print("Number of edges in pattern body: " + swBody.GetEdgeCount());

            if ((swBody == null))
            {
                MessageBox.Show("Select body from 'Solid Bodies' folder");
            }
            else
            {
                swDoc.ClearSelection();
                swOriBody = swBody.GetOriginalPatternedBody(out swMathTrans);

                if ((swOriBody != null))
                {
                    swOriBody.HideBody(true);

                    System.Diagnostics.Debugger.Break();

                    swOriBody.HideBody(false);
                    swOriBody = null;
                }
                swBody = null;
            }
        }

        public SldWorks swApp;
    }
}
```

Get Original Body from Pattern Body (VBA)

This example shows how to get the original body from a pattern body.

```
'-----  
' Preconditions:  
' 1. Open a part document that contains a pattern of solid bodies.  
' 2. Select a pattern body from the Solid Bodies folder.  
'  
' Postconditions:  
' 1. The original body from which the pattern was generated is hidden from view.  
' 2. Click F5 to display the original body.  
'-----  
  
Dim swApp As SldWorks.SldWorks  
Dim swDoc As SldWorks.ModelDoc2  
Dim swSm As SldWorks.SelectionMgr  
Dim selType As Long  
Dim swBody As SldWorks.Body2  
Dim swOriBody As SldWorks.Body2  
Dim swMathTrans As SldWorks.MathTransform  
Option Explicit  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swDoc = swApp.ActiveDoc  
Set swSm = swDoc.SelectionManager  
  
selType = swSm.GetSelectedObjectType3(1, -1)  
Set swBody = swSm.GetSelectedObject6(1, -1)  
  
If (swBody Is Nothing) Then  
    MsgBox ("Select body from 'Solid Bodies' folder")  
Else  
    swDoc.ClearSelection  
    Set swOriBody = swBody.GetOriginalPatternedBody(swMathTrans)  
  
    If Not (swOriBody Is Nothing) Then  
        swOriBody.HideBody True  
  
        Stop  
  
        swOriBody.HideBody False  
        Set swOriBody = Nothing  
    End If  
    Set swBody = Nothing  
End If  
End Sub
```

Get Original Body from Pattern Body (VB.NET)

This example shows how to get the original body from a pattern body.

```

'-----
' Preconditions:
' 1. Open a part document that contains a pattern of solid bodies.
' 2. Select a pattern body from the Solid Bodies folder.
'
' Postconditions:
' 1. The original body from which the pattern was generated is hidden from view.
' 2. Click F5 to display the original body.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

```

    Dim swDoc As ModelDoc2
    Dim swSm As SelectionMgr
    Dim selType As Integer
    Dim swBody As Body2
    Dim swOriBody As Body2
    Dim swMathTrans As MathTransform

    Sub main()

        swDoc = swApp.ActiveDoc
        swSm = swDoc.SelectionManager

        selType = swSm.GetSelectedObjectType3(1, -1)
        swBody = swSm.GetSelectedObject6(1, -1)
        Debug.Print("Number of edges in pattern body: " & swBody.GetEdgeCount)

        If (swBody Is Nothing) Then
            MsgBox("Select body from 'Solid Bodies' folder")
        Else
            swDoc.ClearSelection()
            swOriBody = swBody.GetOriginalPatternedBody(swMathTrans)

            If Not (swOriBody Is Nothing) Then
                swOriBody.HideBody(True)

                Stop

                swOriBody.HideBody(False)
                swOriBody = Nothing
            End If
            swBody = Nothing
        End If

    End Sub

    Public swApp As SldWorks

```

End Class

Get P-Spline Parameterization Data Example (C#)

This example shows how to get P-spline parameter data for a selected spline or curve.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\molds\telephone.sldprt.
// 2. Select a curved edge of Shell1 in the graphics area.
// 3. Open an Immediate Window.
//
// Postconditions:
// 1. Gets the P-spline parameter data for the selected curved edge.
// 2. Inspect the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace GetPSplineParamData_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 Part = null;
            Part = (ModelDoc2)swApp.ActiveDoc;

            SelectionMgr swSelectMgr = null;
            swSelectMgr = (SelectionMgr)Part.SelectionManager;

            Edge swSketchSeg = null;
            swSketchSeg = (Edge)swSelectMgr.GetSelectedObject6(1, -1);

            Curve swCurveIn = null;
            SplineParamData swSplineParaData = default(SplineParamData);
            object varCtrlPoints = null;
            object varKnotPoints = null;
            bool boolStatus = false;
            int i = 0;
            double[] knotPoints;
            double[] ctrlPoints;

            swCurveIn = (Curve)swSketchSeg.GetCurve();
            swSplineParaData = (SplineParamData)swCurveIn.GetPCurveParams2();

            Debug.Print("P-spline");
            Debug.Print("");
            Debug.Print("The dimension is: " + swSplineParaData.Dimension);
            Debug.Print("The order is: " + swSplineParaData.Order);
            Debug.Print("The periodicity is: " + swSplineParaData.Periodic);
            Debug.Print("The knot count is: " + swSplineParaData.KnotPointsCount);
            Debug.Print("Knots: ");
            boolStatus = swSplineParaData.GetKnotPoints(out varKnotPoints);
            knotPoints = (double[])varKnotPoints;
            for (i = 0; i <= knotPoints.GetUpperBound(0); i++)
            {
                Debug.Print(knotPoints[i].ToString());
            }
            Debug.Print("The segment count is: " + swSplineParaData.SegmentCount);
            boolStatus = swSplineParaData.GetSegments(out varCtrlPoints);
            ctrlPoints = (double[])varCtrlPoints;
            Debug.Print((swSplineParaData.SegmentCount * swSplineParaData.Dimension * swSplineParaData.Order) + " Coefficients: ");
            for (i = 0; i <= ctrlPoints.GetUpperBound(0); i++)
            {
                Debug.Print(ctrlPoints[i].ToString());
            }

            Debug.Print("");

            Debug.Print("COLORREF of spline: " + swSplineParaData.Color);
            Debug.Print("Layer of spline: " + swSplineParaData.Layer);
            Debug.Print("Style overrides as defined in swLayerOverride_e: " + swSplineParaData.LayerOverride);
            Debug.Print("Line style as defined in swLineStyles_e: " + swSplineParaData.LineStyle);
            Debug.Print("Line weight as defined in swLineWeights_e: " + swSplineParaData.LineWidth);
        }

        public SldWorks swApp;
    }
}
```

Get P-Spline Parameterization Data Example (VBA)

This example shows how to get P-spline parameter data for a selected spline or curve.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\molds\telephone.sldprt.
' 2. Select a curved edge of Shell1 in the graphics area.
' 3. Open the Immediate Window.
'
' Postconditions:
' 1. Gets the P-spline parameter data for the selected curved edge.
' 2. Inspect the Immediate window.
'-----

```

Option Explicit

```

Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim Part As SldWorks.ModelDoc2

    Set swApp = CreateObject("SldWorks.Application")
    Set Part = swApp.ActiveDoc

    Dim swSelectMgr As Sldworks.SelectionMgr
    Set swSelectMgr = Part.SelectionManager

    Dim swSketchSeg As SldWorks.Edge
    Set swSketchSeg = swSelectMgr.GetSelectedObject6(1, -1)

    Dim swCurveIn As SldWorks.Curve
    Dim swSplineParaData As SldWorks.SplineParamData
    Dim varCtrlPoints As Variant
    Dim varKnotPoints As Variant
    Dim boolStatus As Boolean
    Dim i As Long

    Set swCurveIn = swSketchSeg.GetCurve
    Set swSplineParaData = swCurveIn.GetPCurveParams2

    Debug.Print "P-spline"
    Debug.Print ""
    Debug.Print "The dimension is: " & swSplineParaData.Dimension
    Debug.Print "The order is: " & swSplineParaData.Order
    Debug.Print "The periodicity is: " & swSplineParaData.Periodic
    Debug.Print "The knot count is: " & swSplineParaData.KnotPointsCount
    Debug.Print "Knots: "
    boolStatus = swSplineParaData.GetKnotPoints(varKnotPoints)
    For i = 0 To UBound(varKnotPoints)
        Debug.Print varKnotPoints(i)
    Next i

    Debug.Print "The segment count is: " & swSplineParaData.SegmentCount
    boolStatus = swSplineParaData.GetSegments(varCtrlPoints)
    Debug.Print (swSplineParaData.SegmentCount * swSplineParaData.Dimension * swSplineParaData.Order) & " Coefficients: "
    For i = 0 To UBound(varCtrlPoints)
        Debug.Print varCtrlPoints(i)
    Next i

    Debug.Print ""

    Debug.Print "COLORREF of spline: " & swSplineParaData.Color
    Debug.Print "Layer of spline: " & swSplineParaData.Layer
    Debug.Print "Style overrides as defined in swLayerOverride_e: " & swSplineParaData.LayerOverride
    Debug.Print "Line style as defined in swLineStyles_e: " & swSplineParaData.LineStyle
    Debug.Print "Line weight at defined in swLineWeights_e: " & swSplineParaData.LineWidth

End Sub

```

Get P-Spline Parameterization Data Example (VB.NET)

This example shows how to get P-spline parameter data for a selected spline or curve.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\molds\telephone.sldprt.
' 2. Select a curved edge of Shell1 in the graphics area.
' 3. Open the Immediate Window.
'-----
' Postconditions:
' 1. Gets the P-spline parameter data for the selected curved edge.
' 2. Inspect the Immediate window.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim swApp As SldWorks

        Dim Part As ModelDoc2

        swApp = CreateObject("SldWorks.Application")
        Part = swApp.ActiveDoc

        Dim swSelectMgr As SelectionMgr
        swSelectMgr = Part.SelectionManager

        Dim swSketchSeg As Edge
        swSketchSeg =
swSelectMgr.GetSelectedObject6(1, -1)

        Dim swCurveIn As Curve
        Dim swSplineParaData As SplineParamData
        Dim varCtrlPoints As Object = Nothing
        Dim varKnotPoints As Object = Nothing
        Dim boolStatus As Boolean
        Dim i As Long

        swCurveIn = swSketchSeg.GetCurve
        swSplineParaData = swCurveIn.GetPCurveParams2

        Debug.Print("P-spline")
        Debug.Print("")
        Debug.Print("The dimension is: " & swSplineParaData.Dimension)
        Debug.Print("The order is: " & swSplineParaData.Order)
        Debug.Print("The periodicity is:
" & swSplineParaData.Periodic)

        Debug.Print("The knot count is: " & swSplineParaData.KnotPointsCount)
        Debug.Print("Knots: ")
        boolStatus = swSplineParaData.GetKnotPoints(varKnotPoints)
        For i = 0 To UBound(varKnotPoints)
            Debug.Print(varKnotPoints(i))
        Next i

        Debug.Print("The segment count
is: " & swSplineParaData.SegmentCount)
    
```

```

        boolStatus = swSplineParaData.GetSegments(varCtrlPoints)

        Debug.Print((swSplineParaData.SegmentCount * swSplineParaData.Dimension * swSplineParaData.Order) & " Coefficients: ")

        For i = 0 To UBound(varCtrlPoints)

            Debug.Print(varCtrlPoints(i))

        Next i

        Debug.Print("")

        Debug.Print("COLORREF of spline: " & swSplineParaData.Color)

        Debug.Print("Layer of spline: " & swSplineParaData.Layer)

        Debug.Print("Style overrides as defined in swLayerOverride_e: " & swSplineParaData.LayerOverride)

        Debug.Print("Line style as defined in swLineStyles_e:: " & swSplineParaData.LineStyle)

        Debug.Print("Line weight at defined in swLineWeights_e: " & swSplineParaData.LineWidth)

    End Sub

    Public swApp As SldWorks

End Class

```

Get Parabolas in Sketch Example (VBA)

This example shows how to get information about all of the parabolas in a sketch.

```
'-----
' Preconditions:
' 1. Open a part that contains one or more parabolas.
' 2. Edit the sketch that contains the parabolas.
' 3. Open the Immediate window.
'
' Postconditions: Examine the Immediate window.
'-----
```

Option Explicit

Sub main()

```
Dim swApp      As SldWorks.SldWorks
Dim swModel    As SldWorks.ModelDoc2
Dim swFeat     As SldWorks.feature
Dim swSketch   As SldWorks.sketch
Dim vParabArr  As Variant
Dim i          As Long
Dim bRet       As Boolean
```

```
Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swSketch = swModel.GetActiveSketch2
Set swFeat = swSketch

Debug.Print "File = " & swModel.GetPathName
Debug.Print " " & swFeat.Name
```

vParabArr = swSketch.GetParabolas2

For i = 0 To swSketch.GetParabolaCount - 1

```
Debug.Print "  Parabola[" & i & "]"
Debug.Print "    Color      = " & vParabArr(18 * i + 0)
Debug.Print "    Linetype   = " & vParabArr(18 * i + 1)
```

' Drawings only

```
Debug.Print "    Font       = " & vParabArr(18 * i + 2)
Debug.Print "    Width      = " & vParabArr(18 * i + 3)
Debug.Print "    LayerID    = " & vParabArr(18 * i + 4)
Debug.Print "    Layer Override = " & vParabArr(18 * i + 5)
```

```
Debug.Print "    Start Pt   = (" & vParabArr(18 * i + 6) * 1000# & ", " & vParabArr(18 * i + 7) * 1000# & ", " & vParabArr(18 * i + 8) * 1000# & ") mm"
Debug.Print "    End   Pt   = (" & vParabArr(18 * i + 9) * 1000# & ", " & vParabArr(18 * i + 10) * 1000# & ", " & vParabArr(18 * i + 11) * 1000# & ") mm"
Debug.Print "    Focus Pt  = (" & vParabArr(18 * i + 12) * 1000# & ", " & vParabArr(18 * i + 13) * 1000# & ", " & vParabArr(18 * i + 14) * 1000# & ") mm"
Debug.Print "    Apex  Pt  = (" & vParabArr(18 * i + 15) * 1000# & ", " & vParabArr(18 * i + 16) * 1000# & ", " & vParabArr(18 * i + 17) * 1000# & ") mm"
```

Next i

End Sub

Get Parameters and Spline Points for Curve Example (VBA)

This example shows how to get the parameters and spline points for the selected spline.

```
'-----
' Preconditions:
' 1. Verify that the specified part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Selects a spline in a sketch.
' 3. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----
Option Explicit

' Define two types
Type DoubleRec
    dValue As Double
End Type

Type Long2Rec
    iLower As Long ' Assuming that a C integer has 4 bytes
    iUpper As Long
End Type

' Extract two integer values out of a single double value
' by assigning a DoubleRec to the double value,
' copying the value over an Long2Rec, and
' extracting the integer values
Function ExtractFields(ByVal dValue As Double, iLower As Integer, iUpper As Integer)
    Dim dr As DoubleRec
    Dim i2r As Long2Rec

    ' Set the double value
    dr.dValue = dValue

    ' Copy the values
    LSet i2r = dr

    ' Extract the values
    iLower = i2r.iLower
    iUpper = i2r.iUpper
End Function

Sub DumpModelerSettings(swModeler As SldWorks.Modeler)
    ' Dump modeler settings
    Debug.Print "Modeler Settings:"
    Debug.Print "    BSCurveOutputTol           = " & swModeler.GetToleranceValue(swBSCurveOutputTol)
    Debug.Print "    BSCurveNonRationalOutputTol    = " & swModeler.GetToleranceValue(swBSCurveNonRationalOutputTol)
    Debug.Print "    UVCurveOutputTol               = " & swModeler.GetToleranceValue(swUVCurveOutputTol)
    Debug.Print "    SurfChordTessellationTol        = " & swModeler.GetToleranceValue(swSurfChordTessellationTol)
    Debug.Print "    SurfAngularTessellationTol      = " & swModeler.GetToleranceValue(swSurfAngularTessellationTol)
    Debug.Print "    CurveChordTessellationTol       = " & swModeler.GetToleranceValue(swCurveChordTessellationTol)
    Debug.Print ""
End Sub

Sub DumpBCurveInfo(vBCurveParam As Variant)
    Dim nDim As Integer
    Dim nOrder As Integer
    Dim nCtrlPoints As Integer
    Dim nPeriodicity As Integer
    Dim nNumKnots As Integer
    Dim i As Integer

    ExtractFields vBCurveParam(0), nDim, nOrder
    ExtractFields vBCurveParam(1), nCtrlPoints, nPeriodicity
    nNumKnots = nOrder + nCtrlPoints

    Debug.Print "    Dimension           = " & nDim
    Debug.Print "    Order               = " & nOrder
    Debug.Print "    Number Control Points = " & nCtrlPoints
    Debug.Print "    Periodicity          = " & nPeriodicity
    Debug.Print "    Num Knots           = " & nNumKnots

    Debug.Print ""

    For i = 0 To nNumKnots - 1
        ' Knot weights should be increasing monotonically
        Debug.Print "    Knot(" & i & ")      = " & vBCurveParam(2 + i)
    Next i

    Debug.Print ""

    If 3 = nDim Then
        For i = 2 + nNumKnots To UBound(vBCurveParam) - 1 Step 3
            Debug.Print "    Ctrl(" & (i - 2 - nNumKnots) / 3 & ") = (" & vBCurveParam(i + 0) * 1000# & ", " & vBCurveParam(i + 1) * 1000# & ", " & vBCurveParam(i + 2) * 1000# & ") mm"
        Next i
    Else
        For i = 2 + nNumKnots To UBound(vBCurveParam) - 1 Step 4
            Debug.Print "    Ctrl(" & (i - 2 - nNumKnots) / 4 & ") = (" & vBCurveParam(i + 0) * 1000# & ", " & vBCurveParam(i + 1) * 1000# & ", " &
```

```

vBCurveParam(i + 2) * 1000# & ") mm [" & vBCurveParam(i + 3) & "]"
Next i
End If
End Sub

Sub DumpSplineInfo(vSplinePts As Variant)
Dim i As Long

For i = 0 To UBound(vSplinePts) Step 3
Debug.Print " SplinePt(" & i / 3 & ") = (" & vSplinePts(i + 0) * 1000# & ", " & vSplinePts(i + 1) * 1000# & ", " & vSplinePts(i + 2) * 1000#
& ") mm"
Next i
End Sub

Sub main()

Dim swApp As SldWorks.SldWorks
Dim swModeler As SldWorks.Modeler
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swSketch As SldWorks.Sketch
Dim swSketchSegment As SldWorks.SketchSegment
Dim swCurve As SldWorks.Curve
Dim vBCurveParam As Variant
Dim vSplinePts As Variant
Dim i As Long
Dim bRet As Boolean
Dim fileName As String
Dim errors As Long
Dim warnings As Long

Set swApp = Application.SldWorks
Set swModeler = swApp.GetModeler
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\molds\telephone.sldprt"
Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
Set swModelDocExt = swModel.Extension
Set swSelMgr = swModel.SelectionManager
bRet = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
swModel.EditSketch
swModel.ClearSelection2 True
bRet = swModelDocExt.SelectByID("Spline2", "SKETCHSEGMENT", 6.21547510219904E-02, 0.111861721213803, 5.89464775348876E-02, False, 0, Nothing, 0)
Set swSketchSegment = swSelMgr.GetSelectedObject6(1, -1)
Set swCurve = swSketchSegment.GetCurve

DumpModelerSettings swModeler

vBCurveParam = swCurve.GetBCurveParams(False)
vSplinePts = swCurve.GetSplinePts(vBCurveParam)

Debug.Print "File = " & swModel.GetPathName

DumpBCurveInfo vBCurveParam

Debug.Print " -----"

Debug.Assert Not IsNull(vSplinePts)

DumpSplineInfo vSplinePts

Debug.Print " -----"

End Sub

```

Get Parameters of Reference Axis Example (VBA)

This example shows how to get the parameters of a reference axis.

```
'-----  
' Preconditions:  
' 1. Open a part, assembly, or drawing that  
'    contains a reference axis feature.  
' 2. Select the reference axis feature.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the parameters of the selected  
'    reference axis feature.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swFeat As SldWorks.Feature  
    Dim swRefAxis As SldWorks.RefAxis  
    Dim vAxisParam As Variant  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeat = swSelMgr.GetSelectedObject5(1)  
    Set swRefAxis = swFeat.GetSpecificFeature2  
  
    vAxisParam = swRefAxis.GetRefAxisParams  
  
    Debug.Print "Name = " & swFeat.Name  
    Debug.Print "  Start   = (" & vAxisParam(0) * 1000 & ", " & vAxisParam(1) * 1000 & ", " & vAxisParam(2) * 1000 & ") mm"  
    Debug.Print "  End     = (" & vAxisParam(3) * 1000 & ", " & vAxisParam(4) * 1000 & ", " & vAxisParam(5) * 1000 & ") mm"  
  
End Sub
```

Get Parameters of Conical Surface Example (C#)

This example shows how to get the parameters of a conical surface.

```
//-----
// Preconditions:
// 1. Open a model document with a conical surface.
// 2. Select the conical surface.
// 3. Open an Immediate window.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace ConeParams2_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Face2 swFace = default(Face2);
            Surface swSurf = default(Surface);
            double[] vCone = null;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFace = (Face2)swSelMgr.GetSelectedObject6(1, -1);
            swSurf = (Surface)swFace.GetSurface();

            if (swSurf.IsCone())
            {
                vCone = (double[])swSurf.ConeParams2;
                Debug.Print("Origin          = (" + vCone[0] * 1000.0 + "mm, " + vCone[1] * 1000.0 + "mm, " + vCone[2] * 1000.0 + "mm)");
                Debug.Print("Axis          = (" + vCone[3] + ", " + vCone[4] + ", " + vCone[5] + ")");
                Debug.Print("Radius       = " + vCone[6] * 1000.0 + " mm");
                // 1 radian = 180°/p = 57.295779513° or approximately 57.3°
                Debug.Print("Half angle   = " + vCone[7] * 57.3 + " degrees");
                Debug.Print("Reference direction = (" + vCone[8] * 1000.0 + "mm, " + vCone[9] * 1000.0 + "mm, " + vCone[10] * 1000.0 + "mm)");
            }
        }

        public SldWorks swApp;
    }
}
```

Get Parameters of Conical Surface Example (VBA)

This example shows how to get the parameters of a conical surface.

```
'-----
' Preconditions:
' 1. Open a model document with a conical surface.
' 2. Select the conical surface.
' 3. Open an Immediate window.
'
' Postconditions: Inspect the Immediate window.
'-----
```

Option Explicit

```
Sub main()

    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim swSelMgr              As SldWorks.SelectionMgr
    Dim swFace               As SldWorks.Face2
    Dim swSurf               As SldWorks.Surface
    Dim vCone                As Variant

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)
    Set swSurf = swFace.GetSurface

    If swSurf.IsCone Then
        vCone = swSurf.ConeParams2
        Debug.Print "Origin" = (" & vCone(0) * 1000# & "mm, " & vCone(1) * 1000# & "mm, " & vCone(2) * 1000# & "mm)"
        Debug.Print "Axis" = (" & vCone(3) & ", " & vCone(4) & ", " & vCone(5) & ")
        Debug.Print "Radius" = (" & vCone(6) * 1000# & " mm"
        ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
        Debug.Print "Half angle" = (" & vCone(7) * 57.3 & " degrees"
        Debug.Print "Reference direction" = (" & vCone(8) * 1000# & "mm, " & vCone(9) * 1000# & "mm, " & vCone(10) * 1000# & "mm)"
    End If
End Sub
```

Get Parameters of Conical Surface Example (VB.NET)

This example shows how to get the parameters of a conical surface.

```

'-----
' Preconditions:
' 1. Open a model document with a conical surface.
' 2. Select the conical surface.
' 3. Open an Immediate window.
'-----
' Postconditions: Inspect the Immediate window.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swFace As Face2
        Dim swSurf As Surface
        Dim vCone As Object

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swFace = swSelMgr.GetSelectedObject6(1, -1)
        swSurf = swFace.GetSurface

        If swSurf.IsCone Then
            vCone = swSurf.ConeParams2
            Debug.Print("Origin          = (" & vCone(0) * 1000.0# & "mm, " & vCone(1) * 1000.0# & "mm, " & vCone(2) * 1000.0#
            Debug.Print("Axis            = (" & vCone(3) & ", " & vCone(4) & ", " & vCone(5) & ")")
            Debug.Print("Radius         = " & vCone(6) * 1000.0# & " mm")
            // 1 radian = 180°/p = 57.295779513° or approximately 57.3°
            Debug.Print("Half angle     = " & vCone(7) * 57.3 & " degrees")
            Debug.Print("Reference direction = (" & vCone(8) * 1000.0# & "mm, " & vCone(9) * 1000.0# & "mm, " & vCone(10) * 1000.0#
        End If

    End Sub

    Public swApp As SldWorks

End Class

```

Get Parameters of Cylindrical Surface Example (VBA)

This example shows how to get the origin, axis, and radius for the selected cylindrical surface.

```
'-----  
' Preconditions:  
' 1. Open a part that contains a cylindrical surface.  
' 2. Select the cylindrical surface.  
' 3. Open the Immediate window.  
'  
' Postconditions: Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swFace As SldWorks.Face2  
    Dim swSurf As SldWorks.Surface  
    Dim vCylinder As Variant  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFace = swSelMgr.GetSelectedObject5(1)  
    Set swSurf = swFace.GetSurface  
  
    If swSurf.IsCylinder Then  
        vCylinder = swSurf.CylinderParams  
        Debug.Print "Origin      = (" & vCylinder(0) * 1000# & ", " & vCylinder(1) * 1000# & ", " & vCylinder(2) * 1000# & ") mm"  
        Debug.Print "Axis        = (" & vCylinder(3) & ", " & vCylinder(4) & ", " & vCylinder(5) & ") "  
        Debug.Print "Radius     = " & vCylinder(6) * 1000# & " mm"  
    End If  
  
End Sub
```

Get Parameters of Planar Surface Example (VBA)

This example shows how to get the parameters of a planar surface.

```
'-----  
' Preconditions:  
' 1. Open a part that contains a planar surface.  
' 2. Select the planar surface.  
' 3. Open the Immediate window.  
'  
' Postconditions: Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swFace As SldWorks.Face2  
    Dim swSurf As SldWorks.Surface  
    Dim vPlane As Variant  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFace = swSelMgr.GetSelectedObject5(1)  
    Set swSurf = swFace.GetSurface  
    If swSurf.IsPlane Then  
        vPlane = swSurf.PlaneParams  
        Debug.Print "File = " & swModel.GetPathName  
        Debug.Print "   Normal      = (" & vPlane(0) & ", " & vPlane(1) & ", " & vPlane(2) & ")"  
        Debug.Print "   Root        = (" & vPlane(3) * 1000# & ", " & vPlane(4) * 1000# & ", " & vPlane(5) * 1000# & ") mm"  
    End If  
  
End Sub
```

Get Parameters of Reference Plane Example (VBA)

This example shows how to get the parameters of a reference plane.

```
'-----
' Preconditions:
' 1. Open a model document containing a reference
'    plane.
' 2. Select the reference plane.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the parameters of the selected reference
'    plane.
' 2. Examine the Immediate window.
'
' NOTES:
' * Origin is translation vector from
'   plane origin -> model origin
'   wrt plane coordinate system.
' * Xvector is wrt model coordinate system.
' * Normal is wrt model coordinate system.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swRefPlane As SldWorks.RefPlane
    Dim vPlaneParam As Variant
    Dim nDist As Double
    Dim bRet As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject5(1)
    Set swRefPlane = swFeat.GetSpecificFeature2

    vPlaneParam = swRefPlane.GetRefPlaneParams
    Debug.Print "Plane = " + swFeat.Name

    Debug.Print "  Model Origin (wrt Plane) = (" + Str(vPlaneParam(0) * 1000#) + "," + Str(vPlaneParam(1) * 1000#) + "," + Str(vPlaneParam(2) * 1000#) +
") mm"
    Debug.Print "  Xvector      (wrt Model) = (" + Str(vPlaneParam(3)) + "," + Str(vPlaneParam(4)) + "," + Str(vPlaneParam(5)) + ")"
    Debug.Print "  Normal        (wrt Model) = (" + Str(vPlaneParam(6)) + "," + Str(vPlaneParam(7)) + "," + Str(vPlaneParam(8)) + ")"
    nDist = Sqr(vPlaneParam(0) ^ 2 + vPlaneParam(1) ^ 2 + vPlaneParam(2) ^ 2)
    Debug.Print "  Plane Origin (wrt Model) = (" + Str(nDist * vPlaneParam(6) * 1000#) + "," + Str(nDist * vPlaneParam(7) * 1000#) + "," + Str(nDist *
vPlaneParam(8) * 1000#) + ") mm"

End Sub
```

Get Parameters of Toroidal Surface Example (VBA)

This example shows how to get the parameters of a toroidal surface.

```
'-----  
' Preconditions:  
' 1. Open a part that has a toroidal surface.  
' 2. Select the toroidal surface.  
'  
' Postconditions: Examine the Immediate window.  
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swFace As SldWorks.Face2  
Dim swSurf As SldWorks.Surface  
Dim vTorus As Variant  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swSelMgr = swModel.SelectionManager  
Set swFace = swSelMgr.GetSelectedObject5(1)  
Set swSurf = swFace.GetSurface  
If swSurf.IsTorus Then  
    vTorus = swSurf.TorusParams  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print " center          = (" & vTorus(0) * 1000# & ", " & vTorus(1) * 1000# & ", " & vTorus(2) * 1000# & ") mm"  
    Debug.Print " axis              = (" & vTorus(3) & ", " & vTorus(4) & ", " & vTorus(5) & ") "  
    Debug.Print " major radius    = " & vTorus(6) * 1000# & " mm"  
    Debug.Print " minor radius    = " & vTorus(7) * 1000# & " mm"  
End If
```

End Sub

Get Parent-Child Relationship for Component Example (VBA)

This example shows how to get the parent-child relationship for an assembly component.

```
'-----
' Preconditions:
' 1. Open an assembly.
' 2. Select a component that is not in a subassembly.
'   NOTE: This example does not support selections in
'   subassemblies, because the selected component in
'   a subassembly is not a feature in the top-level assembly.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the parent-child relationship for the selected
'   component.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub ProcessFeature(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swFeat As SldWorks.Feature)

    Dim vParentArr As Variant
    Dim vParent As Variant
    Dim swParentFeat As SldWorks.Feature
    Dim vChildArr As Variant
    Dim vChild As Variant
    Dim swChildFeat As SldWorks.Feature
    Dim i As Long

    vParentArr = swFeat.GetParents
    vChildArr = swFeat.GetChildren

    Debug.Print "   Feature name = " + swFeat.Name + " [" + swFeat.GetType + "]"

    If Not IsEmpty(vParentArr) Then
        Debug.Print "       Parents:"
        For Each vParent In vParentArr
            Set swParentFeat = vParent
            Debug.Print "           " + swParentFeat.Name + " [" + swParentFeat.GetType + "]"
        Next vParent
    End If

    If Not IsEmpty(vChildArr) Then
        Debug.Print "       Children:"
        For Each vChild In vChildArr
            Set swChildFeat = vChild
            Debug.Print "           " + swChildFeat.Name + " [" + swChildFeat.GetType + "]"
        Next vChild
    End If

End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swAssy As SldWorks.AssemblyDoc
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swComp As SldWorks.Component2
    Dim swFeat As SldWorks.Feature
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssy = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swComp = swSelMgr.GetSelectedObjectsComponent4(1, -1)
    Set swFeat = swAssy.FeatureByName(swComp.Name2): Debug.Assert Not swFeat Is Nothing

    Debug.Print "File = " & swModel.GetPathName

    ProcessFeature swApp, swModel, swFeat

End Sub
```

Get Parent Component Example (VBA)

This example shows how to get the parent component of the selected assembly or subassembly component.

```
'-----  
' Preconditions:  
' 1. Open an assembly document.  
' 2. Select an assembly or subassembly component.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the parent component of the selected assembly  
'    or subassembly component.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swConfigMgr As SldWorks.ConfigurationManager  
    Dim swConf As SldWorks.Configuration  
    Dim swComp As SldWorks.Component2  
    Dim swParentComp As SldWorks.Component2  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swComp = swSelMgr.GetSelectedObjectsComponent4(1, -1)  
  
    ' Returns Nothing if selected component is a top-level assembly component  
    Set swParentComp = swComp.GetParent  
    If Nothing Is swParentComp Then  
        Set swConfigMgr = swModel.ConfigurationManager  
        Set swConf = swConfigMgr.ActiveConfiguration  
        Set swParentComp = swConf.GetRootComponent3(True)  
    End If  
  
    Debug.Assert Not swParentComp Is Nothing  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print "   Component = " & swComp.Name2 & " [" & swComp.GetPathName & "]"  
    Debug.Print "   Parent component = " & swParentComp.Name2 & " [" & swParentComp.GetPathName & "]"  
  
End Sub
```

Get Parent Features Example (C++ COM)

This example shows how to get the parent features of the selected feature.

```
// -----
STDMETHODIMP CAPITestATL::API_Test(void)
{
    AFX_MANAGE_STATE(AfxGetStaticModuleState());
    HRESULT          hr = S_OK;
    CComPtr <ISldWorks>    pSldWorks;
    CComPtr <IModelDoc2>    pModel;
    CComPtr <ISelectionMgr>    pSelMgr;
    CComPtr <IUnknown>        pUnk;
    CComQIPtr <IFeature>    pSelFeat;

    long              NumParents = 0;
    long              i = 0;
    IFeature**        pParentsOfFeature = NULL;

    try
    {
        // Connect to SOLIDWORKS Design application
        ASSERT(m_iSldWorks);
        hr = m_iSldWorks->get_IActiveDoc2(&pModel);
        ASSERT(pModel);
        hr = pModel->get_ISelectionManager(&pSelMgr);
        ASSERT(pSelMgr);
        hr = pSelMgr->IGetSelectedObject5(1, &pUnk);
        ASSERT(pUnk);
        pSelFeat = pUnk;
        ASSERT(pSelFeat);
        hr = pSelFeat->IGetParentCount(&NumParents);
        ASSERT(NumParents > 0);
        pParentsOfFeature = new IFeature * [NumParents];
        ZeroMemory(pParentsOfFeature, NumParents * sizeof(IFeature *));
        hr = pSelFeat->IGetParents(pParentsOfFeature);
        // For debugging purposes only
        for (i = 0; i < NumParents; i++)
        {
            CComBSTR    sFeatName;
            CComBSTR    sTypeName;
            hr = pParentsOfFeature[i]->get_Name(&sFeatName);
            hr = pParentsOfFeature[i]->GetTypeName(&sTypeName);
        }
    }
    try
    {
        {
            // Clean up each object in the list
            for (i = 0; i < NumParents; i++)
            {
                if (pParentsOfFeature[i] != NULL)
                {
                    pParentsOfFeature[i]->Release();
                    pParentsOfFeature[i] = NULL;
                }
            }
        }
    }
    catch (...)
    {
        {
            DebugBreak();
        }
    }
}
catch (...)
{
    {
        DebugBreak();
    }
}

delete [] pParentsOfFeature;
return S_OK;
}
// -----
```

Get Part's Feature Statistics Example (C#)

This example shows how to get the statistics of all of the features in a part document.

```
//-----
// Preconditions:
// 1. Open a part that has multiple features.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Gets the statistics for the features in
//    the part.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace GetFeatureStatisticsForPart_CSharp.csproj
{
    partial
    class SolidWorksMacro
    {
        public
        void Main()
        {
            FeatureStatistics swFeatStat = default(FeatureStatistics);
            FeatureManager swFeatMgr = default(FeatureManager);
            ModelDoc2 swModel = default(ModelDoc2);
            String[] featnames = null;
            Int32[] feattypes = null;
            Object[]
features = null;
            Double[] featureUpdateTimes = null;
            Double[] featureUpdatePercentTimes = null;
            int iter = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swFeatMgr = swModel.FeatureManager;
            swFeatStat = swFeatMgr.FeatureStatistics;

            swFeatStat.Refresh();

            Debug.Print("Model name: " + swFeatStat.PartName);
            Debug.Print("
Number of features: " + swFeatStat.FeatureCount);
            Debug.Print("
Number of solid bodies: " + swFeatStat.SolidBodiesCount);
            Debug.Print("
Number of surface bodies: " + swFeatStat.SurfaceBodiesCount);
            Debug.Print("
Total rebuild time: " + swFeatStat.TotalRebuildTime);
            Debug.Print("");
            features
= (Object[])swFeatStat.Features;
            featnames = (String[])swFeatStat.FeatureNames;
            feattypes = (Int32[])swFeatStat.FeatureTypes;
            featureUpdateTimes = (Double[])swFeatStat.FeatureUpdateTimes;
            featureUpdatePercentTimes = (Double[])swFeatStat.FeatureUpdatePercentageTimes;
            if
((featnames != null))
            {
                for
(iter = 0; iter <= featnames.GetUpperBound(0); iter++)
                {
                    Debug.Print("
Feature name: " + featnames[iter]);
                    Debug.Print("
Feature created: " + ((Feature)features[iter]).DateCreated);
                    Debug.Print("
Feature type as defined in sw_SelectType_e: " + feattypes[iter]);
                    Debug.Print("
Update time: " + featureUpdateTimes[iter]);
                    Debug.Print("
Update % time: " + featureUpdatePercentTimes[iter]);
                    Debug.Print("");
                }
            }
        }
        public SldWorks swApp;
    }
}
```

Get Part's Feature Statistics Example (VBA)

This example shows how to get the statistics of all of the features in a part document.

```
'-----
' Preconditions:
' 1. Open a part that has multiple features.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the statistics for the features in
'   the part.
' 2. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swFeatStat As SldWorks.FeatureStatistics
Dim swFeatMgr As SldWorks.FeatureManager
Dim swModel As SldWorks.ModelDoc2
Dim featnames As Variant
Dim feattypes As Variant
Dim features As Variant
Dim featureUpdateTimes As Variant
Dim featureUpdatePercentTimes As Variant
Dim iter As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swFeatMgr = swModel.FeatureManager

    Set swFeatStat = swFeatMgr.FeatureStatistics

    swFeatStat.Refresh

    Debug.Print "Model name:           " & swFeatStat.PartName
    Debug.Print "  Number of features:       " & swFeatStat.FeatureCount
    Debug.Print "  Number of solid bodies:    " & swFeatStat.SolidBodiesCount
    Debug.Print "  Number of surface bodies:  " & swFeatStat.SurfaceBodiesCount
    Debug.Print "  Total rebuild time:        " & swFeatStat.TotalRebuildTime
    featnames = swFeatStat.FeatureNames
    feattypes = swFeatStat.FeatureTypes
    features = swFeatStat.features
    featureUpdateTimes = swFeatStat.featureUpdateTimes
    featureUpdatePercentTimes = swFeatStat.FeatureUpdatePercentageTimes

    If Not (IsEmpty(featnames) Or IsNull(featnames)) Then
        For iter = 0 To UBound(featnames)
            Debug.Print "    Feature name:           " & featnames(iter)
            Debug.Print "    Feature type:           " & feattypes(iter)
            Debug.Print "    Update time:            " & featureUpdateTimes(iter)
            Debug.Print "    Update % time:          " & featureUpdatePercentTimes(iter)
        Next iter
    End If
End Sub
```

Get Part's Feature Statistics Example (VB.NET)

This example shows how to get the statistics of all of the features in a part document.

```
'-----
' Preconditions:
' 1. Open a part that has multiple features.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the statistics for the features in
'    the part.
' 2. Examine the Immediate window.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

Public Sub main()

```
    Dim swFeatStat As FeatureStatistics
    Dim swFeatMgr As FeatureManager
    Dim swModel As ModelDoc2
    Dim featnames As Object
    Dim feattypes As Object
    Dim features As Object
    Dim featureUpdateTimes As Object
    Dim featureUpdatePercentTimes As Object
    Dim iter As Integer

    swModel = swApp.ActiveDoc
    swFeatMgr = swModel.FeatureManager

    swFeatStat = swFeatMgr.FeatureStatistics

    swFeatStat.Refresh()

    Debug.Print("Model name:           " & swFeatStat.PartName)
    Debug.Print("  Number of features:       " & swFeatStat.FeatureCount)
    Debug.Print("  Number of solid bodies:    " & swFeatStat.SolidBodiesCount)
    Debug.Print("  Number of surface bodies:  " & swFeatStat.SurfaceBodiesCount)
    Debug.Print("  Total rebuild time:        " & swFeatStat.TotalRebuildTime)
    featnames = swFeatStat.FeatureNames
    feattypes = swFeatStat.FeatureTypes
    features = swFeatStat.features
    featureUpdateTimes = swFeatStat.featureUpdateTimes
    featureUpdatePercentTimes = swFeatStat.FeatureUpdatePercentageTimes
    If Not featnames Is Nothing Then
        For iter = 0 To UBound(featnames)
            Debug.Print("    Feature name:           " & featnames(iter))
            Debug.Print("    Feature type:           " & feattypes(iter))
            Debug.Print("    Update time:            " & featureUpdateTimes(iter))
            Debug.Print("    Update % time:          " & featureUpdatePercentTimes(iter))
        Next iter
    End If
```

End Sub

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

End Class

Get Part Bounding Box Example (VBA)

This example shows how to get an accurate bounding box for a open part.

```
'-----  
'  
' Problem:  
' IPartDoc::GetPartBox returns an approximate  
' bounding box and cannot be relied upon to be used  
' in calculations. Typically, the bounding box  
' returned is larger and not a minimal bounding box.  
' The best use for IPartDoc::GetPartbox is as a first  
' approximation to determine whether two parts are intersecting.  
'  
' If there is a need for a more accurate bounding  
' box, then use one based on the display tessellation, which  
' gives a more minimal bounding box.  
'  
' Preconditions: Part is open.  
'  
' Postconditions: None  
'  
'-----
```

Option Explicit

```
Function GetMax _  
( _  
    Val1 As Double, _  
    Val2 As Double, _  
    Val3 As Double, _  
    Val4 As Double _  
) As Double  
' Finds maximum of four values  
    GetMax = Val1  
  
    If Val2 > GetMax Then  
        GetMax = Val2  
    End If  
    If Val3 > GetMax Then  
        GetMax = Val3  
    End If  
    If Val4 > GetMax Then  
        GetMax = Val4  
    End If  
End Function  
Function GetMin _  
( _  
    Val1 As Double, _  
    Val2 As Double, _  
    Val3 As Double, _  
    Val4 As Double _  
) As Double  
' Finds minimum of four values  
    GetMin = Val1  
  
    If Val2 < GetMin Then  
        GetMin = Val2  
    End If  
    If Val3 < GetMin Then  
        GetMin = Val3  
    End If  
    If Val4 < GetMin Then  
        GetMin = Val4  
    End If  
End Function  
Sub ProcessTessTriangles _
```

```

( _
    vTessTriangles As Variant, _
    X_max As Double, _
    X_min As Double, _
    Y_max As Double, _
    Y_min As Double, _
    Z_max As Double, _
    Z_min As Double _
)

Dim i As Long
For i = 0 To UBound(vTessTriangles) / (1 * 9) - 1
' Debugging output only
' Debug.Print "Pt(" + Str(i) + ") = "
' Debug.Print "(" + _
'     Str(vTessTriangles(9 * i + 0)) + "," + _
'     Str(vTessTriangles(9 * i + 1)) + "," + _
'     Str(vTessTriangles(9 * i + 2)) + ")"
' Debug.Print "(" + _
'     Str(vTessTriangles(9 * i + 3)) + "," + _
'     Str(vTessTriangles(9 * i + 4)) + "," + _
'     Str(vTessTriangles(9 * i + 5)) + ")"
' Debug.Print "(" + _
'     Str(vTessTriangles(9 * i + 6)) + "," + _
'     Str(vTessTriangles(9 * i + 7)) + "," + _
'     Str(vTessTriangles(9 * i + 8)) + ")"

X_max = GetMax((vTessTriangles(9 * i + 0)), (vTessTriangles(9 * i + 3)), (vTessTriangles(9 * i + 6)), X_max)
X_min = GetMin((vTessTriangles(9 * i + 0)), (vTessTriangles(9 * i + 3)), (vTessTriangles(9 * i + 6)), X_min)

Y_max = GetMax((vTessTriangles(9 * i + 1)), (vTessTriangles(9 * i + 4)), (vTessTriangles(9 * i + 7)), Y_max)
Y_min = GetMin((vTessTriangles(9 * i + 1)), (vTessTriangles(9 * i + 4)), (vTessTriangles(9 * i + 7)), Y_min)

Z_max = GetMax((vTessTriangles(9 * i + 2)), (vTessTriangles(9 * i + 5)), (vTessTriangles(9 * i + 8)), Z_max)
Z_min = GetMin((vTessTriangles(9 * i + 2)), (vTessTriangles(9 * i + 5)), (vTessTriangles(9 * i + 8)), Z_min)
Next i
End Sub

Sub ProcessBodies _
( _
    vBodies As Variant, _
    X_max As Double, _
    X_min As Double, _
    Y_max As Double, _
    Y_min As Double, _
    Z_max As Double, _
    Z_min As Double _
)

Dim swBody As SldWorks.body2
Dim swFace As SldWorks.face2
Dim vTessTriangles As Variant
Dim i As Long

' Probably empty if no reference surfaces
If IsEmpty(vBodies) Then Exit Sub

For i = 0 To UBound(vBodies)
    Set swBody = vBodies(i)
    Set swFace = swBody.GetFirstFace
    While Not swFace Is Nothing
        vTessTriangles = swFace.GetTessTriangles(True)
        ProcessTessTriangles vTessTriangles, X_max, X_min, Y_max, Y_min, Z_max, Z_min

        Set swFace = swFace.GetNextFace
    Wend
Next i
End Sub

Sub main()
    Const MaxDouble As Double = 1.79769313486231E+308
    Const MinDouble As Double = -1.79769313486231E+308
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc
    Dim vBodies As Variant

```

```
Dim vBoundingBox As Variant
Dim X_max As Double
Dim X_min As Double
Dim Y_max As Double
Dim Y_min As Double
Dim Z_max As Double
Dim Z_min As Double
Dim i As Long
```

```
Set swApp = CreateObject("SldWorks.Application")
Set swModel = swApp.ActiveDoc
Set swPart = swModel
```

```
' Initialise to large/small values
```

```
X_max = MinDouble
X_min = MaxDouble
Y_max = MinDouble
Y_min = MaxDouble
Z_max = MinDouble
Z_min = MaxDouble
```

```
' Solid body
```

```
vBodies = swPart.GetBodies2(swSolidBody, False)
ProcessBodies vBodies, X_max, X_min, Y_max, Y_min, Z_max, Z_min
```

```
' Reference surfaces
```

```
vBodies = swPart.GetBodies2(swSheetBody, False)
ProcessBodies vBodies, X_max, X_min, Y_max, Y_min, Z_max, Z_min
```

```
' Approximate bounding box
```

```
vBoundingBox = swPart.GetPartBox(True)
Debug.Print "Tessellation Quality = " & Str(swModel.GetTessellationQuality)
Debug.Print ""
```

```
Debug.Print "PartBox = "
```

```
Debug.Print " (" & _
    Str(vBoundingBox(0) * 1000#) & "," & _
    Str(vBoundingBox(1) * 1000#) & "," & _
    Str(vBoundingBox(2) * 1000#) & ") mm"
```

```
Debug.Print " (" & _
    Str(vBoundingBox(3) * 1000#) & "," & _
    Str(vBoundingBox(4) * 1000#) & "," & _
    Str(vBoundingBox(5) * 1000#) & ") mm"
```

```
Debug.Print ""
```

```
Debug.Print "TessBox = "
```

```
Debug.Print " (" & _
    Str(X_min * 1000#) & "," & _
    Str(Y_min * 1000#) & "," & _
    Str(Z_min * 1000#) & ") mm"
```

```
Debug.Print " (" & _
    Str(X_max * 1000#) & "," & _
    Str(Y_max * 1000#) & "," & _
    Str(Z_max * 1000#) & ") mm"
```

```
Debug.Print ""
```

```
End Sub
```

```
'-----
```

Get Part for Corresponding Component Example (VBA)

This example shows how to get the names of the part entity and the corresponding assembly component entity.

```

'-----
'
' Preconditions:
'   (1) Assembly document is open.
'   (2) Select the feature whose part's and component's name you want.
'
' Postconditions: None
'
'-----

Option Explicit

Sub main()
    Dim swApp          As SldWorks.SldWorks
    Dim swModel         As SldWorks.ModelDoc2
    Dim swModelDocComp  As SldWorks.ModelDoc2
    Dim swModelDocExt   As SldWorks.ModelDocExtension
    Dim swSelMgr        As SldWorks.SelectionMgr
    Dim swAssyFeat       As SldWorks.feature
    Dim swPartFeat       As SldWorks.feature
    Dim swComp           As SldWorks.Component2
    Dim vFaceArr         As Variant
    Dim swAssyFace        As SldWorks.face2
    Dim swPartFace        As SldWorks.face2
    Dim swAssyEnt         As SldWorks.entity
    Dim swPartEnt         As SldWorks.entity
    Dim bRet              As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager

    Set swAssyFeat = swSelMgr.GetSelectedObject5(1)
    Set swComp = swSelMgr.GetSelectedObjectsComponent2(1)
    Set swModelDocComp = swComp.GetModelDoc
    Set swModelDocExt = swModelDocComp.Extension

    vFaceArr = swAssyFeat.GetFaces: Debug.Assert Not IsEmpty(vFaceArr)

    Set swAssyFace = vFaceArr(0)
    Set swAssyEnt = swAssyFace
    Set swPartEnt = swModelDocExt.GetCorrespondingEntity(swAssyEnt): Debug.Assert Not swPartEnt Is Nothing
    Set swPartFace = swPartEnt
    Set swPartFeat = swPartFace.GetFeature: Debug.Assert Not swPartFeat Is Nothing

    Debug.Print "Comp      = " + swComp.Name2 + " [" + swComp.GetPathName + "]"
    Debug.Print " AssyFeat  = " + swAssyFeat.Name
    Debug.Print " PartFeat   = " + swPartFeat.Name

End Sub
'-----

```

Get Paths and Titles of Parents of Virtual Component (C#)

This example shows how to get the paths and titles of the parent assembly components of a virtual component.

```
//-----
// Preconditions:
// 1. Open:
//    public_documents\samples\tutorial\api\assem2.sldasm
// 2. Open the Immediate window.
//
// Postconditions: The paths and titles of the parent
// assembly components, up to and including the first non-virtual
// parent assembly component, are printed in the Immediate Window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;
using System;

namespace IsVirtualComponent3CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 doc = default(ModelDoc2);
            doc = (ModelDoc2)swApp.ActiveDoc;
            if (doc == null)
                return;
            if (doc.GetType() != (int)swDocumentTypes_e.swDocASSEMBLY)
                return;

            AssemblyDoc asm = default(AssemblyDoc);
            asm = (AssemblyDoc)doc;

            object[] components = null;
            components = (object[])asm.GetComponents(false);
            // Get all components

            if (components != null)
            {
                Debug.Print("Virtual components in this assembly:");

                foreach (Object compt in components)
                {
                    Component2 comp = default(Component2);
                    comp = (Component2)compt;

                    ModelDoc2 compDoc = default(ModelDoc2);
                    compDoc = (ModelDoc2)comp.GetModelDoc2();
                    if ((compDoc != null))
                    {
                        bool result3 = false;
                        object pathChain = null;
                        object titleChain = null;
                        result3 = compDoc.Extension.IsVirtualComponent3(out pathChain, out titleChain);

                        object[] pathChainArray = null;
                        object[] titleChainArray = null;
                        pathChainArray = (object[])pathChain;
                        titleChainArray = (object[])titleChain;

                        if (result3 != false)
                        {
                            Debug.Print(" Virtual component name: " + comp.Name2);

                            Debug.Print("    " + "Paths:");
                            foreach (Object path in pathChainArray)
                            {
                                Debug.Print("        " + path);
                            }

                            Debug.Print("    " + "Titles:");
                            foreach (Object title in titleChainArray)
                            {
                                Debug.Print("        " + title);
                            }
                        }
                    }
                    else
                    {
                        Debug.Print(comp.Name2 + " <<< NOT LOADED >>>");
                    }
                }
            }
        }
    }
}
```

```
/// <summary>  
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>
```

```
public SldWorks swApp;
```

```
}  
}
```

Get Paths and Titles of Parents of Virtual Component (VBA)

This example shows how to get the paths and titles of the parent assembly components of a virtual component.

```

'-----
' Preconditions:
' 1. Open:
'    public_documents\samples\tutorial\api\assem2.sldasm
' 2. Open the Immediate window.
'
' Postconditions: The paths and titles of the parent
' assembly components, up to and including the first non-virtual
' parent assembly component, are printed in the Immediate Window.
'-----
Option Explicit

Sub main()
    Dim swApp As SldWorks.SldWorks
    Set swApp = Application.SldWorks

    Dim doc As ModelDoc2
    Set doc = swApp.ActiveDoc
    If doc Is Nothing Then Exit Sub
    If doc.GetType <> swDocASSEMBLY Then Exit Sub

    Dim asm As AssemblyDoc
    Set asm = doc

    Dim components As Variant
    components = asm.GetComponents(False) ' Get all components

    If IsArray(components) Then
        Debug.Print "Virtual components in this assembly:"

        Dim vComp As Variant
        For Each vComp In components
            Dim comp As Component2
            Set comp = vComp

            Dim compDoc As ModelDoc2
            Set compDoc = comp.GetModelDoc2
            If Not compDoc Is Nothing Then
                Dim bResult3 As Boolean
                Dim pathChain As Variant
                Dim titleChain As Variant
                bResult3 = compDoc.Extension.IsVirtualComponent3(pathChain, titleChain)

                If bResult3 <> False Then
                    Debug.Print "  Virtual component name: " & comp.Name2

                    Debug.Print "    " & "Paths:"
                    Dim vPath As Variant
                    For Each vPath In pathChain
                        Debug.Print "      " & vPath
                    Next

                    Debug.Print "    " & "Titles:"
                    Dim vTitle As Variant
                    For Each vTitle In titleChain
                        Debug.Print "      " & vTitle
                    Next
                End If
            Else
                Debug.Print comp.Name2 & " <<< NOT LOADED >>>"
            End If
        Next
    End If
End Sub

```

Get Paths and Titles of Parents of Virtual Component (VB.NET)

This example shows how to get the paths and titles of the parent assembly components of a virtual component.

```

'-----
' Preconditions:
' 1. Open:
'    public_documents\samples\tutorial\api\assem2.sldasm
' 2. Open the Immediate window.
'
' Postconditions: The paths and titles of the parent
' assembly components, up to and including the first non-virtual
' parent assembly component, are printed in the Immediate Window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Diagnostics
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim doc As ModelDoc2
        doc = swApp.ActiveDoc
        If doc Is Nothing Then Exit Sub
        If doc.GetType <> swDocumentTypes_e.swDocASSEMBLY Then Exit Sub

        Dim asm As AssemblyDoc
        asm = doc

        Dim components As Object
        components = asm.GetComponents(False) ' Get all components

        If IsArray(components) Then
            Debug.Print("Virtual components in this assembly:")

            Dim compt As Object
            For Each compt In components
                Dim comp As Component2
                comp = compt

                Dim compDoc As ModelDoc2
                compDoc = comp.GetModelDoc2
                If Not compDoc Is Nothing Then
                    Dim result3 As Boolean
                    Dim pathChain As Object = Nothing
                    Dim titleChain As Object = Nothing
                    result3 = compDoc.Extension.IsVirtualComponent3(pathChain, titleChain)

                    If result3 <> False Then
                        Debug.Print("  Virtual component name: " & comp.Name2)

                        Debug.Print("    " & "Paths:")
                        Dim path As Object
                        For Each path In pathChain
                            Debug.Print("      " & path)
                        Next

                        Debug.Print("    " & "Titles:")
                        Dim title As Object
                        For Each title In titleChain
                            Debug.Print("      " & title)
                        Next
                    End If
                Else
                    Debug.Print(comp.Name2 & " <<< NOT LOADED >>>")
                End If
            Next
        End If

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Get Paths of Open Documents (C#)

This example shows how to get an array of the open documents, and their paths and filenames, in the current SOLIDWORKS Design session.

```
//-----  
// Preconditions: At least one model document is  
//               open in SOLIDWORKS Design.  
//  
// Postconditions: None.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
using System.Runtime.InteropServices;  
namespace GetOpenDocPaths_CSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
        ModelDoc2 swModel;  
        object[] models;  
        int count;  
        int index;  
        string pathName;  
        public void Main()  
        {  
            count = swApp.GetDocumentCount();  
            Debug.Print("Number of open documents in this SOLIDWORKS Design session: " + count);  
            models = (object[])swApp.GetDocuments();  
            for (index = 0; index < count; index++)  
            {  
                swModel = models[index] as ModelDoc2;  
                pathName = swModel.GetPathName();  
                Debug.Print("Path and name of open document: " + pathName);  
            }  
        }  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
        public SldWorks swApp;  
    }  
}
```

Get Paths of Open Documents (VBA)

This example shows how to get an array of the open documents, and their paths and filenames, in the current SOLIDWORKS Design session.

```
' -----  
' Preconditions: At least one document is  
'               open in SOLIDWORKS Design.  
'  
' Postconditions: None.  
' -----  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim vModels As Variant  
Dim count As Long  
Dim index As Long  
Sub main()  
Set swApp = Application.SldWorks  
count = swApp.GetDocumentCount  
Debug.Print "Number of open documents in this SolidWork Design session: " & count  
vModels = swApp.GetDocuments  
For index = LBound(vModels) To UBound(vModels)  
    Set swModel = vModels(index)  
    Debug.Print "Path and name of open document: " & swModel.GetPathName  
    Next index  
End Sub
```

Get Paths of Open Documents (VB.NET)

This example shows how to get an array of the open documents, and their paths and filenames, in the current SOLIDWORKS Design session.

```
' -----  
' Preconditions: At least one document is  
'               open in SOLIDWORKS Design.  
'  
' Postconditions: None.  
' -----  
  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Dim swModel As ModelDoc2  
    Dim models As Object  
    Dim count As Integer  
    Dim index As Integer  
  
    Public Sub main()  
  
        count = swApp.GetDocumentCount  
        Debug.Print("Number of open documents in this SOLIDWORKS Design session: " & count)  
        models = swApp.GetDocuments  
        For index = LBound(models) To UBound(models)  
            swModel = models(index)  
            Debug.Print("Path and name of open document: " & swModel.GetPathName)  
        Next index  
  
    End Sub  
  
    ''' <summary>  
    ''' The SldWorks swApp variable is pre-assigned for you.  
    ''' </summary>  
    Public swApp As SldWorks  
  
End Class
```

Get Pattern Display Dimension Example (C#)

This example shows how to get the display dimensions for various pattern properties of a linear pattern.

```
//-----
// Preconditions:
// 1. Open a part with a linear pattern.
// 2. Select the linear pattern feature.
// 3. Open the Immediate window.
//
// Postconditions: Inspect the Immediate window.
// -----

using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetDisplayDimension_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        SelectionMgr selMan;
        Feature feat;
        DisplayDimension dispDim;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            selMan = (SelectionMgr)Part.SelectionManager;
            feat = (Feature)selMan.GetSelectedObject6(1, -1);
            Debug.Print(" Feature = " + feat.Name);

            dispDim = (DisplayDimension)feat.GetDisplayDimension((int)swFeatureDimensionParameter_e.swPatternInstanceCount1);
            Debug.Print(" Instance count 1 display dimension = " + dispDim.GetNameForSelection());
            Debug.Print(" Type of dimension as defined in swDimensionType_e = " + dispDim.Type2);
            dispDim = (DisplayDimension)feat.GetDisplayDimension((int)swFeatureDimensionParameter_e.swPatternSpacing1);
            Debug.Print(" Spacing 1 display dimension = " + dispDim.GetNameForSelection());
        }

        public SldWorks swApp;
    }
}
```

Get Pattern Display Dimension Example (VBA)

This example shows how to get the display dimensions for various pattern properties of a linear pattern.

```
'-----  
' Preconditions:  
' 1. Open a part with a linear pattern.  
' 2. Select the linear pattern feature.  
' 3. Open the Immediate window.  
'  
' Postconditions: Inspect the Immediate window.  
'-----  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim selMan As SldWorks.SelectionMgr  
Dim feat As SldWorks.Feature  
Dim dispDim As SldWorks.DisplayDimension  
  
Option Explicit  
Sub main()  
    Set swApp = Application.SldWorks  
  
    Set Part = swApp.ActiveDoc  
    Set selMan = Part.SelectionManager  
    Set feat = selMan.GetSelectedObject6(1, -1)  
    Debug.Print " Feature = " & feat.Name  
  
    Set dispDim = feat.GetDisplayDimension(swFeatureDimensionParameter_e.swPatternInstanceCount1)  
    Debug.Print " Instance count 1 display dimension = " & dispDim.GetNameForSelection  
    Debug.Print " Type of dimension as defined in swDimensionType_e = " & dispDim.Type2  
    Set dispDim = feat.GetDisplayDimension(swFeatureDimensionParameter_e.swPatternSpacing1)  
    Debug.Print " Spacing 1 display dimension = " & dispDim.GetNameForSelection  
    Debug.Print " Type of dimension as defined in swDimensionType_e = " & dispDim.Type2  
  
End Sub
```

Get Pattern Display Dimension Example (VB.NET)

This example shows how to get the display dimensions for various pattern properties of a linear pattern.

```
'-----
' Preconditions:
' 1. Open a part with a linear pattern.
' 2. Select the linear pattern feature.
' 3. Open the Immediate window.
'
' Postconditions: Inspect the Immediate window.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
    Dim selMan As SelectionMgr
    Dim feat As Feature
    Dim dispDim As DisplayDimension
```

```
    Sub main()
```

```
        Part = swApp.ActiveDoc
        selMan = Part.SelectionManager
        feat = selMan.GetSelectedObject6(1, -1)
        Debug.Print(" Feature = " & feat.Name)
```

```
        dispDim = feat.GetDisplayDimension(swFeatureDimensionParameter_e.swPatternInstanceCount1)
        Debug.Print(" Instance count 1 display dimension = " & dispDim.GetNameForSelection)
        Debug.Print(" Type of dimension as defined in swDimensionType_e = " & dispDim.Type2)
        dispDim = feat.GetDisplayDimension(swFeatureDimensionParameter_e.swPatternSpacing1)
        Debug.Print(" Spacing 1 display dimension = " & dispDim.GetNameForSelection)
        Debug.Print(" Type of dimension as defined in swDimensionType_e = " & dispDim.Type2)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Get Patterned Cosmetic Thread Annotations Data Example (C#)

This example shows how to get data for patterned cosmetic thread annotations in a drawing.

```
//-----
// Preconditions:
// 1. Open a drawing that contains at least one
// drawing view with patterned cosmetic thread annotations.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Iterates through the drawing views and prints
// to the Immediate window the patterned cosmetic
// thread annotations data in each drawing view.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace CThreadCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            MathUtility swMathUtility = default(MathUtility);
            ModelDoc2 myModel = default(ModelDoc2);
            DrawingDoc myDrawing = default(DrawingDoc);
            View thisView = default(View);
            CThread thisCThread = default(CThread);

            swMathUtility = (MathUtility)swApp.GetMathUtility();
            myModel = (ModelDoc2)swApp.ActiveDoc;
            myDrawing = (DrawingDoc)myModel;
            myModel.ClearSelection2(true);

            //Traverse annotations of the drawing views in this drawing and look for cosmetic threads
            thisView = (View)myDrawing.GetFirstView();
            while ((thisView != null))
            {
                if (thisView.Name == "Sheet1")
                {
                    Debug.Print("");
                }
                else
                {
                    Debug.Print("View name = " + thisView.Name);
                    thisCThread = (CThread)thisView.GetFirstCThread();
                    while ((thisCThread != null))
                    {
                        processCosmeticThread(myModel, thisCThread);
                        thisCThread = (CThread)thisCThread.GetNext();
                    }
                }
                thisView = (View)thisView.GetNextView();
            }
        }

        private void processCosmeticThread(ModelDoc2 myModel, CThread aCThread)
        {
            Annotation cthreadAnno = default(Annotation);
            string annoName = null;
            string annoVis = null;
            int patternedCount = 0;
            object[] vPatternedXform = null;
            int i = 0;
            MathTransform transform = default(MathTransform);
            double[] vTransform = null;
            cthreadAnno = (Annotation)aCThread.GetAnnotation();
            annoName = cthreadAnno.GetName();
            if ((cthreadAnno.Visible == (int)swAnnotationVisibilityState_e.swAnnotationHidden))
            {
                annoVis = "Hidden";
            }
            else
            {
                annoVis = "Visible";
            }
            Debug.Print(" Processing CThread " + annoName + "(" + annoVis + ")");

            //Retrieve information about any patterns made from this cosmetic thread
            patternedCount = aCThread.GetPatternedTransformsCount();
            Debug.Print(" Pattern count = " + patternedCount);
            vPatternedXform = (object[])aCThread.PatternedTransforms;
            if ((vPatternedXform != null))
            {
                for (i = 0; i < vPatternedXform.Length; i++)
                {
                    transform = (MathTransform)vPatternedXform[i];
                    vTransform = (double[])transform.ArrayData;
                    if ((vTransform != null))
                    {
                        Debug.Print(" Rotate (" + vTransform[0].ToString("###0.0#####") + " " + vTransform[1].ToString("###0.0#####") + " " + vTransform[2].ToString("###0.0#####");
                        Debug.Print(" " + vTransform[3].ToString("###0.0#####") + " " + vTransform[4].ToString("###0.0#####") + " " + vTransform[5].ToString("###0.0#####");
                        Debug.Print(" " + vTransform[6].ToString("###0.0#####") + " " + vTransform[7].ToString("###0.0#####") + " " + vTransform[8].ToString("###0.0#####") + ")");
                        Debug.Print(" Translate " + vTransform[9].ToString("###0.0#####") + " " + vTransform[10].ToString("###0.0#####") + " " + vTransform[11].ToString("###0.0#####");
                        Debug.Print("");
                    }
                }
            }
        }
    }
}
```

```
    /// <summary>
    ///     The Sldworks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
```

Get Patterned Cosmetic Thread Annotations Data Example (VBA)

This example shows how to get data for patterned cosmetic thread annotations in a drawing.

```

'-----
' Preconditions:
' 1. Open a drawing that contains at least one
'    drawing view with patterned cosmetic thread annotations.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Iterates through the drawing views and prints
'    to the Immediate window the patterned cosmetic
'    thread annotations data in each drawing view.
' 2. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swMathUtility As SldWorks.MathUtility

Sub main()

    Dim myModel As SldWorks.ModelDoc2
    Dim myDrawing As SldWorks.DrawingDoc
    Dim thisView As SldWorks.View
    Dim thisCThread As SldWorks.CThread

    Set swApp = Application.SldWorks
    Set swMathUtility = swApp.GetMathUtility()
    Set myModel = swApp.ActiveDoc
    Set myDrawing = myModel
    myModel.ClearSelection2 True

    ' Traverse annotations of the drawing views in this drawing and look for cosmetic threads
    Set thisView = myDrawing.GetFirstView()
    While Not thisView Is Nothing
        If thisView.Name = "Sheet1" Then
            Debug.Print
        Else
            Debug.Print "View name = " & thisView.Name
            Set thisCThread = thisView.GetFirstCThread()
            While Not thisCThread Is Nothing
                Call processCosmeticThread(myModel, thisCThread)
                Set thisCThread = thisCThread.GetNext()
            Wend
        End If
        Set thisView = thisView.GetNextView()
    Wend
End Sub

Private Sub processCosmeticThread(myModel As SldWorks.ModelDoc2, aCThread As SldWorks.CThread)
    Dim cthreadAnno As SldWorks.Annotation
    Dim annoName As String, annoVis As String
    Dim patternedCount As Long
    Dim vPatternedXform As Variant
    Dim i As Integer
    Dim transform As SldWorks.MathTransform
    Dim vTransform As Variant

    Set cthreadAnno = aCThread.GetAnnotation()
    annoName = cthreadAnno.GetName()
    If (cthreadAnno.Visible = SwConst.swAnnotationHidden) Then
        annoVis = "Hidden"
    Else
        annoVis = "Visible"
    End If

    Debug.Print "  Processing CThread " & annoName & "(" & annoVis & ")"

    ' Retrieve information about any patterns made from this cosmetic thread
    patternedCount = aCThread.GetPatternedTransformsCount()
    Debug.Print "    Pattern count = " & patternedCount
    vPatternedXform = aCThread.PatternedTransforms()
    If Not IsEmpty(vPatternedXform) Then
        For i = LBound(vPatternedXform) To UBound(vPatternedXform)
            Set transform = vPatternedXform(i)
            vTransform = transform.ArrayData()
            If Not IsEmpty(vTransform) Then
                Debug.Print "      Rotate (" & Format(vTransform(0), "###0.0#####") & " " & Format(vTransform(1), "###0.0#####") & " " &
                    Format(vTransform(2), "###0.0#####")
                Debug.Print "          " & Format(vTransform(3), "###0.0#####") & " " & Format(vTransform(4), "###0.0#####") & " " &
                    Format(vTransform(5), "###0.0#####")
                Debug.Print "          " & Format(vTransform(6), "###0.0#####") & " " & Format(vTransform(7), "###0.0#####") & " " &
                    Format(vTransform(8), "###0.0#####") & ")"
                Debug.Print "      Translate " & Format(vTransform(9), "###0.0#####") & " " & Format(vTransform(10), "###0.0#####") & " " &
                    Format(vTransform(11), "###0.0#####")
                Debug.Print
            End If
        Next i
    End If
End Sub

```

Get Patterned Cosmetic Thread Annotations Data Example (VB.NET)

This example shows how to get data for patterned cosmetic thread annotations in a drawing.

```

-----
Preconditions:
1. Open a drawing that contains at least one
   drawing view with patterned cosmetic thread annotations.
2. Open the Immediate window.
Postconditions:
1. Iterates through the drawing views and prints
   to the Immediate window the patterned cosmetic
   thread annotations data in each drawing view.
2. Examine the Immediate window.
-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swMathUtility As MathUtility
        Dim myModel As ModelDoc2
        Dim myDrawing As DrawingDoc
        Dim thisView As View
        Dim thisCThread As CThread

        swMathUtility = swApp.GetMathUtility()
        myModel = swApp.ActiveDoc
        myDrawing = myModel
        myModel.ClearSelection2(True)

        'Traverse annotations of the drawing views in this drawing and look for cosmetic threads
        thisView = myDrawing.GetFirstView()
        While Not thisView Is Nothing
            If thisView.Name = "Sheet1" Then
                Debug.Print("")
            Else
                Debug.Print("View name = " & thisView.Name)
                thisCThread = thisView.GetFirstCThread()
                While Not thisCThread Is Nothing
                    Call processCosmeticThread(myModel, thisCThread)
                    thisCThread = thisCThread.GetNext()
                End While
            End If
            thisView = thisView.GetNextView()
        End While
    End Sub

    Private Sub processCosmeticThread(ByVal myModel As ModelDoc2, ByVal aCThread As CThread)
        Dim cthreadAnno As Annotation
        Dim annoName As String, annoVis As String
        Dim patternedCount As Integer
        Dim vPatternedXform As Object
        Dim i As Integer
        Dim transform As MathTransform
        Dim vTransform As Object
        cthreadAnno = aCThread.GetAnnotation()
        annoName = cthreadAnno.GetName()
        If (cthreadAnno.Visible = swAnnotationVisibilityState_e.swAnnotationHidden) Then
            annoVis = "Hidden"
        Else
            annoVis = "Visible"
        End If
        Debug.Print(" Processing CThread " & annoName & "(" & annoVis & ")")

        'Retrieve information about any patterns made from this cosmetic thread
        patternedCount = aCThread.GetPatternedTransformsCount()
        Debug.Print(" Pattern count = " & patternedCount)
        vPatternedXform = aCThread.PatternedTransforms()
        If Not vPatternedXform Is Nothing Then
            For i = LBound(vPatternedXform) To UBound(vPatternedXform)
                transform = vPatternedXform(i)
                vTransform = transform.ArrayData()
                If Not vTransform Is Nothing Then
                    Debug.Print(" Rotate (" & Format(vTransform(0), "###0.0#####") & " " & Format(vTransform(1), "###0.0#####") & " " & Format(vTransform(2), "###0.0#####"))
                    Debug.Print(" " & Format(vTransform(3), "###0.0#####") & " " & Format(vTransform(4), "###0.0#####") & " " & Format(vTransform(5), "###0.0#####"))
                    Debug.Print(" " & Format(vTransform(6), "###0.0#####") & " " & Format(vTransform(7), "###0.0#####") & " " & Format(vTransform(8), "###0.0#####") & ")")
                    Debug.Print(" Translate " & Format(vTransform(9), "###0.0#####") & " " & Format(vTransform(10), "###0.0#####") & " " & Format(vTransform(11), "###0.0#####"))
                    Debug.Print("")
                End If
            Next i
        End If
    End Sub

    '--- <summary>
    '--- The SldWorks swApp variable is pre-assigned for you.
    '--- </summary>
    Public swApp As SldWorks

End Class

```

Get Persistent Identifiers and Type for Sketch Points Example (VBA)

This example shows how to get the persistent identifiers and types for sketch points.

NOTE: SOLIDWORKS Design allocates a persistent ID for sketch points and segments, accessible by `ISketchPoint::GetID`. This method allows you to store the identifier and then return to the sketch entity at a later time. There are also sketch points that are not visible to the user. These are typically used internally by SOLIDWORKS Design for various purposes. Sketch points are also created from other operations; for example, creating a spline or adding a midpoint relation. Each sketch point has a read-only property, `ISketchPoint::Type`, that indicates how it is used in the sketch.

```

'-----
' Preconditions:
' 1. Open a part or assembly.
' 2. Select a 2D or 3D sketch.
'
' Postconditions:
' 1. Gets the selected sketch's sketch-point IDs and types.
' 2. Examine the Immediate window.
'
' NOTES:
' * Both SketchPoint::ID and ISketchPoint::Type are read-only.
' * The identifier is unique to the sketch and may be duplicated
'   in the model. To unambiguously resolve a sketch entity, you need both
'   the sketch and the sketch identifier.
'-----

```

Option Explicit

```

Sub main()
    Dim swApp           As SldWorks.SldWorks
    Dim swModel          As SldWorks.ModelDoc2
    Dim swSelMgr          As SldWorks.SelectionMgr
    Dim swFeat           As SldWorks.Feature
    Dim swSketch         As SldWorks.Sketch
    Dim vSketchPtArr     As Variant
    Dim vSketchPt        As Variant
    Dim swSketchPt       As SldWorks.SketchPoint
    Dim vID              As Variant

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swSketch = swFeat.GetSpecificFeature

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " " & swFeat.Name

    vSketchPtArr = swSketch.GetSketchPoints: If IsEmpty(vSketchPtArr) Then Exit Sub

    For Each vSketchPt In vSketchPtArr
        Set swSketchPt = vSketchPt
        vID = swSketchPt.GetID
        Debug.Print "    Pt [" & vID(0) & ", " & vID(1) & "] = (" & swSketchPt.X * 1000# & ", " & swSketchPt.Y * 1000# & ", " &
        swSketchPt.Z * 1000# & ") mm"
        Debug.Print "    Type = " & swSketchPt.Type
    Next vSketchPt
End Sub

```

Get Persistent Identifiers and Types for Sketch Points Example (C#)

This example shows how to get the persistent identifiers and types for sketch points.

NOTE: SOLIDWORKS Design allocates a persistent ID for sketch points and segments, accessible by `ISketchPoint::GetID`. This method allows you to store the identifier and then return to the sketch entity at a later time. There are also sketch points that are not visible to the user. These are typically used internally by SOLIDWORKS Design for various purposes. Sketch points are also created from other operations; for example, creating a spline or adding a midpoint relation. Each sketch point has a read-only property, `ISketchPoint::Type`, that indicates how it is used in the sketch.

```
//-----
// Preconditions:
// 1. Open a part or assembly.
// 2. Select a 2D or 3D sketch.
//
// Postconditions:
// 1. Gets the selected sketch's sketch-point IDs and types.
// 2. Examine the Immediate window.
//
// NOTES:
// * Both SketchPoint::ID and ISketchPoint::Type are read-only.
// * The identifier is unique to the sketch and may be duplicated
//   in the model. To unambiguously resolve a sketch entity, you need both
//   the sketch and the sketch identifier.
//-----

using System.Diagnostics;
using SolidWorks.Interop.sldworks;

namespace Macro1
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            SelectionMgr swSelMgr;
            Feature swFeat;
            Sketch swSketch;
            object[] vSketchPtArr;
            SketchPoint swSketchPt;
            int[] vID;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swSketch = (Sketch)swFeat.GetSpecificFeature();

            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" " + swFeat.Name);

            vSketchPtArr = (object[])swSketch.GetSketchPoints();

            foreach (var vSketchPt in vSketchPtArr)
            {
                swSketchPt = (SketchPoint)vSketchPt;
                vID = (int[])swSketchPt.GetID();
                Debug.Print("    Pt [" + vID[0] + ", " + vID[1] + "] = (" + swSketchPt.X * 1000.0 + ", " + swSketchPt.Y * 1000.0 +
                    ", " + swSketchPt.Z * 1000.0 + ") mm");
                Debug.Print("    Type = " + swSketchPt.Type);
            }
        }

        public SldWorks swApp;
    }
}
```

Get Persistent Identifiers and Type for Sketch Points Example (VB.NET)

This example shows how to get the persistent identifiers and types for sketch points.

NOTE: SOLIDWORKS Design allocates a persistent ID for sketch points and segments, accessible by `ISketchPoint::GetID`. This method allows you to store the identifier and then return to the sketch entity at a later time. There are also sketch points that are not visible to the user. These are typically used internally by SOLIDWORKS Design for various purposes. Sketch points are also created from other operations; for example, creating a spline or adding a midpoint relation. Each sketch point has a read-only property, `ISketchPoint::Type`, that indicates how it is used in the sketch.

```
'-----
' Preconditions:
' 1. Open a part or assembly.
' 2. Select a 2D or 3D sketch.
'
' Postconditions:
' 1. Gets the selected sketch's sketch-point IDs and types.
' 2. Examine the Immediate window.
'
' NOTES:
' * Both SketchPoint::ID and ISketchPoint::Type are read-only.
' * The identifier is unique to the sketch and may be duplicated
'   in the model. To unambiguously resolve a sketch entity, you need both
'   the sketch and the sketch identifier.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim swSketch As Sketch
        Dim vSketchPtArr As Object
        Dim vSketchPt As Object
        Dim swSketchPt As SketchPoint
        Dim vID As Object
```

```
        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swSketch = swFeat.GetSpecificFeature
```

```
        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" " & swFeat.Name)
```

```
        vSketchPtArr = swSketch.GetSketchPoints : If IsNothing(vSketchPtArr) Then Exit Sub
```

```
        For Each vSketchPt In vSketchPtArr
            swSketchPt = vSketchPt
            vID = swSketchPt.GetID
            Debug.Print("    Pt [" & vID(0) & ", " & vID(1) & "] = (" & swSketchPt.X * 1000.0# & ", " & swSketchPt.Y * 1000.0# &
            Debug.Print("    Z * 1000.0# & ") mm")
            Debug.Print("    Type = " & swSketchPt.Type)
        Next vSketchPt
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Get Plane On Which Sketch Created Example (VBA)

This example shows how to get the plane on which the sketch used for the feature was created.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\cstick.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the plane on which the sketch for Revolve1
'    was created.
' 2. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeat As SldWorks.Feature
Dim boolstatus As Boolean
Dim longstatus As Long
Dim parents As Variant
Dim swParentFeat As SldWorks.Feature
Dim swSketch As SldWorks.Sketch
Dim swSketchPlane As Object
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    boolstatus = swModel.Extension.SelectByID2("Revolve1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)
    Set swFeat = swSelMgr.GetSelectedObject5(1)
    parents = swFeat.GetParents
    For i = 0 To UBound(parents)
        Set swParentFeat = parents(i)
        If swParentFeat.GetTypeName = "ProfileFeature" Then
            Set swSketch = swParentFeat.GetSpecificFeature2
            Set swSketchPlane = swSketch.GetReferenceEntity(longstatus)
            'The plane can be either a face or a Feature object
            Debug.Print ("Type of reference entity (4 = swSelectType_e.swSelDATUMPLANES): " & longstatus)
        End If
    Next i

    swModel.ClearSelection2 True

End Sub
```

Get Plane or Face for Sketch Example (VBA)

This example shows how to get the plane or face on which a selected sketch was drawn.

```
'-----
' Preconditions:
' 1. Open apart or assembly.
' 2. Select a sketch.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Selects the plane or face on which the selected
'    sketch was drawn.
' 2. Examine the Immediate window and graphics area.
'-----
Option Explicit

Sub main()

    Dim pSWApp As SldWorks.SldWorks
    Dim pModel As SldWorks.ModelDoc2
    Dim pSelMgr As SldWorks.SelectionMgr
    Dim pFeat As SldWorks.Feature
    Dim pSketch As SldWorks.Sketch
    Dim pEntity As SldWorks.Entity
    Dim pPlaneFeat As SldWorks.Feature
    Dim pFace As SldWorks.Face2
    Dim pRefPlane As SldWorks.RefPlane
    Dim vPlaneParams As Variant
    Dim vNormal As Variant
    Dim bRet As Boolean
    Dim nEntityType As Long

    Set pSWApp = CreateObject("SldWorks.Application")
    Set pModel = pSWApp.ActiveDoc
    Set pSelMgr = pModel.SelectionManager
    Set pFeat = pSelMgr.GetSelectedObject5(1)
    Set pSketch = pFeat.GetSpecificFeature2
    Set pEntity = pSketch.GetReferenceEntity(nEntityType)

    Debug.Print "Sketch Name = " + pFeat.Name

    If nEntityType = swSelDATUMPLANES Then
        Set pPlaneFeat = pEntity
        Set pRefPlane = pEntity
        vPlaneParams = pRefPlane.GetRefPlaneParams
        Debug.Print "Plane Selected!"
        Debug.Print "  Origin   = (" + Str(vPlaneParams(0) * 1000#) + "," + Str(vPlaneParams(1) * 1000#) + "," + Str(vPlaneParams(2) * 1000#) + ") mm"
        Debug.Print "  Xvector  = (" + Str(vPlaneParams(3)) + "," + Str(vPlaneParams(4)) + "," + Str(vPlaneParams(5)) + ")"
        Debug.Print "  Normal   = (" + Str(vPlaneParams(6)) + "," + Str(vPlaneParams(7)) + "," + Str(vPlaneParams(8)) + ")"
        ' Cannot select through Entity; must
        ' select through Feature
        bRet = pPlaneFeat.Select2(False, 0)
    End If

    If nEntityType = swSelFACES Then
        Set pFace = pEntity
        vNormal = pFace.Normal
        Debug.Print "Face Selected!"
        Debug.Print "  Normal   = (" + Str(vNormal(0)) + "," + Str(vNormal(1)) + "," + Str(vNormal(2)) + ")"
        bRet = pEntity.Select2(False, 0)
    End If

End Sub
```

Get PMI Data Example (C#)

This example shows how to get Product and Manufacturing Information (PMI) from a Step 242 file.

```
// -----
// Preconditions:
// 1. Open a Step 242 file that contains PMI annotations.
// 2. Open an Immediate window.
// 3. Add breakpoints as needed and press F10 to walk through the macro.
//
// Postconditions:
// Inspect the Immediate window for PMI annotations, if available.
// -----
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Reflection;
using System.Runtime.CompilerServices;
using System.Security;
using System.Text;
using System.Threading.Tasks;
using Microsoft.VisualBasic;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace GetPMIData_CSharp
{
    partial class SolidWorksMacro
    {
        private ModelDoc2 swModel;
        private ModelDocExtension swModelDocExt;
        private Annotation swAnnotation;
        private PMIDatumData swPMIDatumData;
        private PMIDatumFeature swPMIDatumFeature;
        private PMIDatumTarget swPMIDatumTarget;
        private PMIDimensionData swPMIDataDim;
        private PMIDimensionItem swPMIDataDimItem;
        private PMIGtolData swPMIGtolData;
        private PMIGtolBoxData swPMIGtolBoxData;
        private PMIFrameData swPMIFrameData;
        private PMIGtolFrameDatum swPMIGtolFrameDatum;
        private int IAnnoPMIType;
        private int iAnnoCnt;
        private object[] arrAnnotation;
        private int i;
        private int j;
        private int k;
        private int l;
        private int IAnnoType;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;

            iAnnoCnt = swModelDocExt.GetAnnotationCount();

            if (iAnnoCnt > 0)
            {
                arrAnnotation = (object[])swModelDocExt.GetAnnotations();

                for (i = 0; i <= arrAnnotation.GetUpperBound(0); i++)
                {
                    swAnnotation = (Annotation)arrAnnotation[i];

                    IAnnoType = swAnnotation.GetType();

                    IAnnoPMIType = swAnnotation.GetPMIType();

                    switch (IAnnoType)
                    {
                        case 1:
                        {
                            Debug.Print("Annotation: Thread");
                            break;
                        }

                        case 2:
                        {
                            Debug.Print("Annotation: Datum Tag " + swAnnotation.GetName());

                            if (IAnnoPMIType == 2)
                            {

```

```

        swPMIDatumData = (PMIDatumData)swAnnotation.GetPMIData();

        Debug.Print("PMI datum data");

        if (swPMIDatumData.GetDatumType() == (int)swPMIDatumType_e.swPMIDatumType_DatumFeature)
        {

            // Get IPMIDatumFeature

            swPMIDatumFeature = (PMIDatumFeature)swPMIDatumData.GetDatumFeature();
            Debug.Print(" Datum feature");
            Debug.Print(" Label: " + swPMIDatumFeature.Label);
            Debug.Print(" Leader anchor style as defined in swPMIDatumAnchorStyle_e: " + swPMIDatumFeature.LeaderAnchorStyle_e);
            Debug.Print(" Leader bend length: " + swPMIDatumFeature.LeaderBendLength);
            Debug.Print(" Datum shape as defined in swPMIDatumShape_e: " + swPMIDatumFeature.Shape);
            Debug.Print(" Datum text: " + swPMIDatumFeature.Text);
        }

        if (swPMIDatumData.GetDatumType() == (int)swPMIDatumType_e.swPMIDatumType_DatumTarget)
        {

            // Get IPMIDatumTarget

            swPMIDatumTarget = (PMIDatumTarget)swPMIDatumData.GetDatumTarget();

            Debug.Print(" Datum target");
            Debug.Print(" Area style as defined in swPMIDatumTargetAreaStyle_e: " + swPMIDatumTarget.AreaStyle_e);
            Debug.Print(" Diameter: " + swPMIDatumTarget.Diameter);
            Debug.Print(" Height: " + swPMIDatumTarget.Height);
            Debug.Print(" Movable style as defined in swPMIDatumTargetMovableStyle_e: " + swPMIDatumTarget.MovableStyle_e);
            Debug.Print(" Symbol style as defined in swPMIDatumTargetSymbolStyle_e: " + swPMIDatumTarget.SymbolStyle_e);
            Debug.Print(" Width: " + swPMIDatumTarget.Width);
        }

        Debug.Print("");
    }
    else if (IAnnoPMIType == 0)
    {
        if (swPMIDatumData == null)
            Debug.Print(" No PMI data");
        else
            Debug.Print(" PMIDatumData object returned In Error");
    }
    else
        Debug.Print(" Wrong PMI data type");
    break;
}

case 3:
{
    Debug.Print("Annotation: Datum Target Symbol");
    break;
}

case 4:
{
    Debug.Print("Annotation: Display Dimension " + swAnnotation.GetName());

    if (IAnnoPMIType == 1)
    {
        swPMIDataDim = (PMIDimensionData)swAnnotation.GetPMIData();

        if (swPMIDataDim != null)
        {
            for (k = 0; k <= swPMIDataDim.GetDimensionItemCount() - 1; k++)
            {
                swPMIDataDimItem = (PMIDimensionItem)swPMIDataDim.GetDimensionItemAtIndex(k);

                Debug.Print(" Dimension item " + k);
                Debug.Print(" Text: " + swPMIDataDimItem.DimensionText);
                Debug.Print(" Instance count: " + swPMIDataDimItem.InstanceCount);
                Debug.Print(" Tolerance type as defined in swTolType_e: " + swPMIDataDimItem.TolType);
                Debug.Print(" Value: " + swPMIDataDimItem.Value);
                Debug.Print(" Value precision: " + swPMIDataDimItem.ValuePrecision);
                Debug.Print(" Units: " + swPMIDataDimItem.Unit);
                Debug.Print(" Minimum tolerance: " + swPMIDataDimItem.MinVariation);
                Debug.Print(" Maximum tolerance: " + swPMIDataDimItem.MaxVariation);
                Debug.Print(" Tolerance precision: " + swPMIDataDimItem.TolerancePrecision);
                Debug.Print(" Symbol as defined in swDimensionSymbol_e: " + swPMIDataDimItem.Symbol);
                Debug.Print(" Additional symbol as defined in swToleranceZoneModifier_e: " + swPMIDataDimItem.AdditionalSymbol);
                Debug.Print(" Prefix: " + swPMIDataDimItem.Prefix);
                Debug.Print(" Suffix: " + swPMIDataDimItem.Suffix);
                Debug.Print(" Callout: " + swPMIDataDimItem.CalloutText);
                Debug.Print("");
            }
        }
    }
    else if ((IAnnoPMIType == 0))
    {
        if (swPMIDataDim == null)
            Debug.Print(" No PMI data");
    }
}

```

```

        else
            Debug.Print("    PMIDimensionData object returned in error");
    }
    else
        Debug.Print("    Wrong PMI data type");
    break;
}

case 5:
{
    Debug.Print("Annotation: Gtol " + swAnnotation.GetName());

    if (IAnnoPMIType == 3)
    {
        swPMIDataGtol = (PMIGtolData)swAnnotation.GetPMIData();

        if (swPMIDataGtol != null)
        {
            Debug.Print("    Is composite? " + swPMIDataGtol.IsComposite());
            Debug.Print("    Text: " + swPMIDataGtol.GetText());
            Debug.Print("    Text below frames: " + swPMIDataGtol.GetTextBelowFrames());
            Debug.Print("    Instance count: " + swPMIDataGtol.InstanceCount);
            Debug.Print("    Leader location as defined in swPMILeaderLocation_e: " + swPMIDataGtol.LeaderLoca
            Debug.Print("    Leader modifier as defined in swPMILeaderModifier_e: " + swPMIDataGtol.LeaderModi
            Debug.Print("    Leader style as defined in swPMILeaderStyle_e: " + swPMIDataGtol.LeaderStyle);
            Debug.Print("    Leader type as defined in swPMILeaderType_e: " + swPMIDataGtol.LeaderType);

            int iFrameCount;
            iFrameCount = swPMIDataGtol.GetFrameCount();

            Debug.Print("    Frame count is " + iFrameCount);

            if (iFrameCount == 0)
                Debug.Print("Gtol frame count is zero - API error");

            for (j = 0; j <= swPMIDataGtol.GetFrameCount() - 1; j++)
            {
                swPMIFrameData = (PMIFrameData)swPMIDataGtol.GetFrameAtIndex(j);
                if (swPMIFrameData != null)
                {
                    Debug.Print("    Gtol frame " + j + " data");
                    Debug.Print("        Geometric characteristic as defined in swGeometricCharacteristic_e: " +
                    Debug.Print("        Frame number: " + swPMIFrameData.FrameNumber);

                    // Get IPMIGtolBoxData

                    for (k = 0; k <= swPMIFrameData.GetGtolBoxCount() - 1; k++)
                    {
                        swPMIGtolBoxData = (PMIGtolBoxData)swPMIFrameData.GetGtolBoxAtIndex(k);
                        Debug.Print("        Gtol box " + k);
                        Debug.Print("        Material modifier as defined in swMaterialModifier_e: " + swPMIGt
                        Debug.Print("        Tolerance: " + swPMIGtolBoxData.TolValue);
                        Debug.Print("        Units as defined in swPMIUnit_e: " + swPMIGtolBoxData.Unit);
                        Debug.Print("        Tolerance zone modifier as defined in swToleranceZoneModifier_e:
                        Debug.Print("        Tolerance per unit area type as defined in swPMITolPerUnitAreaTyp
                        Debug.Print("        Tolerance 1 per unit area value: " + swPMIGtolBoxData.TolerancePe
                        Debug.Print("        Tolerance 2 per unit area value: " + swPMIGtolBoxData.TolerancePe
                    }

                    // Get IPMIGtolFrameDatum

                    for (l = 0; l <= swPMIFrameData.GetGtolDatumCount() - 1; l++)
                    {
                        swPMIGtolFrameDatum = (PMIGtolFrameDatum)swPMIFrameData.GetGtolDatumAtIndex(l);
                        Debug.Print("        Gtol frame datum " + l);
                        Debug.Print("        Datum: " + swPMIGtolFrameDatum.Datum);
                        Debug.Print("        Material modifier as defined in swMaterialModifier_e: " + swPMIGt
                        Debug.Print("        Material modifier value if present: " + swPMIGtolFrameDatum.Datum
                        Debug.Print("        First linked datum: " + swPMIGtolFrameDatum.DatumLinked1);
                        Debug.Print("        Material modifier of first linked datum as defined in swMaterial
                        Debug.Print("        Value, if present, of material modifier of first linked datum: "
                        Debug.Print("        Second linked datum: " + swPMIGtolFrameDatum.DatumLinked2);
                        Debug.Print("        Material modifier of second linked datum as defined in swMaterial
                        Debug.Print("        Value, if present, of material modifier of second linked datum: '
                    }
                }
            }
        }
        else
            Debug.Print("Gtol frame retrieval error");
    }
}

else if (IAnnoPMIType == 0)
{
    if (swPMIDataGtol == null)
        Debug.Print("    No PMI data");
    else
        Debug.Print("    PMIGtolData object returned in error");
}
else
    Debug.Print("    Wrong PMI data type");
break;
}

case 6:
{

```

```

        Debug.Print("Annotation: Note");
        break;
    }

case 7:
{
    Debug.Print("Annotation: SF Symbol");
    break;
}

case 8:
{
    Debug.Print("Annotation: Weld Symbol");
    break;
}

case 9:
{
    Debug.Print("Annotation: Custom Symbol");
    break;
}

case 10:
{
    Debug.Print("Annotation: Dowel Symbol");
    break;
}

case 11:
{
    Debug.Print("Annotation: Leader");
    break;
}

case 12:
{
    Debug.Print("Annotation: Block");
    break;
}

case 13:
{
    Debug.Print("Annotation: Centermark symbol");
    break;
}

case 14:
{
    Debug.Print("Annotation: Table");
    break;
}

case 15:
{
    Debug.Print("Annotation: Centerline");
    break;
}

case 16:
{
    Debug.Print("Annotation: Datum Origin");
    break;
}
    }
}
}

}

/// <summary>
///     ''' The SldWorks swApp variable is pre-assigned for you.
///     ''' </summary>
public SldWorks swApp;
}

}

```

Get PMI Data Example (VBA)

This example shows how to get Product and Manufacturing Information (PMI) from a Step 242 file.

```

'-----
' Preconditions:
' 1. Open a Step 242 file that contains PMI annotations.
' 2. Open an Immediate window.
' 3. Add breakpoints as needed and press F8 to walk through the macro.
'
' Postconditions:
' Inspect the Immediate window for PMI annotations, if available.
'-----

Option Explicit
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swAnnotation As SldWorks.Annotation
Dim swPMIDatumData As SldWorks.PMIDatumData
Dim swPMIDatumFeature As SldWorks.PMIDatumFeature
Dim swPMIDatumTarget As SldWorks.PMIDatumTarget
Dim swPMIDataDim As SldWorks.PMIDimensionData
Dim swPMIDataDimItem As SldWorks.PMIDimensionItem
Dim swPMIDataGtol As SldWorks.PMIGtolData
Dim swPMIGtolBoxData As SldWorks.PMIGtolBoxData
Dim swPMIFrameData As SldWorks.PMIFrameData
Dim swPMIGtolFrameDatum As SldWorks.PMIGtolFrameDatum
Dim IAnnoPMIType As Long
Dim iAnnoCnt As Long
Dim arrAnnotation As Variant
Dim dText As String
Dim i As Long
Dim j As Long
Dim k As Long
Dim l As Long
Dim IAnnoType As Long

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    iAnnoCnt = swModelDocExt.GetAnnotationCount()

    If iAnnoCnt > 0 Then
        arrAnnotation = swModelDocExt.GetAnnotations()

        For i = LBound(arrAnnotation) To UBound(arrAnnotation)

            Set swAnnotation = arrAnnotation(i)

            IAnnoType = swAnnotation.GetType()

            IAnnoPMIType = swAnnotation.GetPMIType

            Select Case IAnnoType
                Case 1
                    Debug.Print "Annotation: Thread"

                Case 2
                    Debug.Print "Annotation: Datum Tag " & swAnnotation.GetName

                    If IAnnoPMIType = 2 Then

                        Set swPMIDatumData = swAnnotation.GetPMIData

                        Debug.Print "PMI datum data"

                        dText = ""

                        If swPMIDatumData.GetDatumType = SwConst.swPMIDatumType_e.swPMIDatumType_DatumFeature Then
                            dText = "swPMIDatumType_e.swPMIDatumType_DatumFeature"

                            'Get IPMIDatumFeature

                            Set swPMIDatumFeature = swPMIDatumData.GetDatumFeature
                            Debug.Print " Datum feature"
                            Debug.Print " Label: " & swPMIDatumFeature.Label
                            Debug.Print " Leader anchor style as defined in swPMIDatumAnchorStyle_e: " &

```

```

swPMIDatumFeature.LeaderAnchorStyle
    Debug.Print " Leader bend length: " & swPMIDatumFeature.LeaderBendLength
    Debug.Print " Datum shape as defined in swPMIDatumShape_e: " & swPMIDatumFeature.Shape
    Debug.Print " Datum text: " & swPMIDatumFeature.Text
End If

If swPMIDatumData.GetDatumType = SwConst.swPMIDatumType_e.swPMIDatumType_DatumTarget Then
    dText = "swPMIDatumType_e.swPMIDatumType_DatumTarget"

    'Get IPMIDatumTarget
    Set swPMIDatumTarget = swPMIDatumData.GetDatumTarget

    Debug.Print " Datum target"
    Debug.Print " Area style as defined in swPMIDatumTargetAreaStyle_e: " & swPMIDatumTarget.AreaStyle
    Debug.Print " Diameter: " & swPMIDatumTarget.Diameter
    Debug.Print " Height: " & swPMIDatumTarget.Height
    Debug.Print " Movable style as defined in swPMIDatumTargetMovableStyle_e: " &
swPMIDatumTarget.MovableStyle
    Debug.Print " Symbol style as defined in swPMIDatumTargetSymbolStyle_e: " &
swPMIDatumTarget.SymbolStyle
    Debug.Print " Width: " & swPMIDatumTarget.Width
End If

Debug.Print ""

ElseIf IAnnoPMIType = 0 Then
    If swPMIDatumData Is Nothing Then
        Debug.Print " No PMI data"
    Else
        Debug.Print " PMIDatumData object returned In Error"
    End If
Else
    Debug.Print " Wrong PMI data type"
End If

Case 3
    Debug.Print "Annotation: Datum Target Symbol"
Case 4
    Debug.Print "Annotation: Display Dimension " & swAnnotation.GetName

    If IAnnoPMIType = 1 Then
        Set swPMIDataDim = swAnnotation.GetPMIData

        If Not swPMIDataDim Is Nothing Then

            For k = 0 To swPMIDataDim.GetDimensionItemCount - 1
                Set swPMIDataDimItem = swPMIDataDim.GetDimensionItemAtIndex(k)

                Debug.Print " Dimension item " & k
                Debug.Print " Text: " & swPMIDataDim.DimensionText
                Debug.Print " Instance count: " & swPMIDataDimItem.InstanceCount
                Debug.Print " Tolerance type as defined in swTolType_e: " & swPMIDataDimItem.TolType
                Debug.Print " Value: " & swPMIDataDimItem.Value
                Debug.Print " Value precision: " & swPMIDataDimItem.ValuePrecision
                Debug.Print " Units: " & swPMIDataDimItem.Unit
                Debug.Print " Minimum tolerance: " & swPMIDataDimItem.MinVariation
                Debug.Print " Maximum tolerance: " & swPMIDataDimItem.MaxVariation
                Debug.Print " Tolerance precision: " & swPMIDataDimItem.TolerancePrecision
                Debug.Print " Symbol as defined in swDimensionSymbol_e: " & swPMIDataDimItem.Symbol
                Debug.Print " Additional symbol as defined in swToleranceZoneModifier_e: " &
swPMIDataDimItem.AdditionalSymbol
                Debug.Print " Prefix: " & swPMIDataDimItem.Prefix
                Debug.Print " Suffix: " & swPMIDataDimItem.Suffix
                Debug.Print " Callout: " & swPMIDataDimItem.CalloutText
                Debug.Print ""
            Next
        End If

    ElseIf (IAnnoPMIType = 0) Then
        If swPMIDataDim Is Nothing Then
            Debug.Print " No PMI data"
        Else
            Debug.Print " PMIDimensionData object returned in error"
        End If
    Else
        Debug.Print " Wrong PMI data type"
    End If
End If
Case 5
    Debug.Print "Annotation: Gtol " & swAnnotation.GetName

    If IAnnoPMIType = 3 Then

        Set swPMIDataGtol = swAnnotation.GetPMIData

```

If Not swPMIDataGtol Is Nothing Then

```
Debug.Print " Is composite? " & swPMIDataGtol.IsComposite
Debug.Print " Text: " & swPMIDataGtol.GetText
Debug.Print " Text below frames: " & swPMIDataGtol.GetTextBelowFrames
Debug.Print " Instance count: " & swPMIDataGtol.InstanceCount
Debug.Print " Leader location as defined in swPMILeaderLocation_e: " & swPMIDataGtol.LeaderLocation
Debug.Print " Leader modifier as defined in swPMILeaderModifier_e: " & swPMIDataGtol.LeaderModifier
Debug.Print " Leader style as defined in swPMILeaderStyle_e: " & swPMIDataGtol.LeaderStyle
Debug.Print " Leader type as defined in swPMILeaderType_e: " & swPMIDataGtol.LeaderType
```

```
Dim iFrameCount As Integer
iFrameCount = swPMIDataGtol.GetFrameCount
```

```
Debug.Print " Frame count is " & iFrameCount
```

```
If iFrameCount = 0 Then
    Debug.Print "Gtol frame count is zero - API error"
End If
```

```
For j = 0 To swPMIDataGtol.GetFrameCount - 1
```

```
    Set swPMIFrameData = swPMIDataGtol.GetFrameAtIndex(j)
    If Not swPMIFrameData Is Nothing Then
```

```
        Debug.Print " Gtol frame " & j & " data"
        Debug.Print " Geometric characteristic as defined in swGeometricCharacteristic_e: " &
swPMIFrameData.GeometricCharacteristic
        Debug.Print " Frame number: " & swPMIFrameData.FrameNumber
```

```
        'Get IPMIGtolBoxData
```

```
        For k = 0 To swPMIFrameData.GetGtolBoxCount - 1
            Set swPMIGtolBoxData = swPMIFrameData.GetGtolBoxAtIndex(k)
            Debug.Print " Gtol box " & k
            Debug.Print " Material modifier as defined in swMaterialModifier_e: " &
swPMIGtolBoxData.MaterialModifier
            Debug.Print " Tolerance: " & swPMIGtolBoxData.TolValue
            Debug.Print " Units as defined in swPMIUnit_e: " & swPMIGtolBoxData.Unit
            Debug.Print " Tolerance zone modifier as defined in swToleranceZoneModifier_e: " &
swPMIGtolBoxData.ToleranceZoneModifier
            Debug.Print " Tolerance per unit area type as defined in swPMITolPerUnitAreaType_e: " &
swPMIGtolBoxData.TolerancePerUnitType
            Debug.Print " Tolerance 1 per unit area value: " & swPMIGtolBoxData.TolerancePerUnitValue1
            Debug.Print " Tolerance 2 per unit area value: " & swPMIGtolBoxData.TolerancePerUnitValue2
        Next
```

```
        'Get IPMIGtolFrameDatum
```

```
        For l = 0 To swPMIFrameData.GetGtolDatumCount - 1
            Set swPMIGtolFrameDatum = swPMIFrameData.GetGtolDatumAtIndex(l)
            Debug.Print " Gtol frame datum " & l
            Debug.Print " Datum: " & swPMIGtolFrameDatum.Datum
            Debug.Print " Material modifier as defined in swMaterialModifier_e: " &
swPMIGtolFrameDatum.DatumModifier
            Debug.Print " Material modifier value if present: " &
swPMIGtolFrameDatum.DatumModifierValue
            Debug.Print " First linked datum: " & swPMIGtolFrameDatum.DatumLinked1
            Debug.Print " Material modifier of first linked datum as defined in swMaterialModifier_e:
" & swPMIGtolFrameDatum.DatumLinkedModifier1
            Debug.Print " Value, if present, of material modifier of first linked datum: " &
swPMIGtolFrameDatum.DatumLinkedModifierValue1
            Debug.Print " Second linked datum: " & swPMIGtolFrameDatum.DatumLinked2
            Debug.Print " Material modifier of second linked datum as defined in swMaterialModifier_e:
" & swPMIGtolFrameDatum.DatumLinkedModifier2
            Debug.Print " Value, if present, of material modifier of second linked datum: " &
swPMIGtolFrameDatum.DatumLinkedModifierValue2
        Next
        Else
            Debug.Print "Gtol frame retrieval error"
        End If
```

```
    Next
```

```
End If
```

```
ElseIf IAnnoPMIType = 0 Then
    If swPMIDataGtol Is Nothing Then
        Debug.Print " No PMI data"
    Else
        Debug.Print " PMIGtolData object returned in error"
    End If
Else
    Debug.Print " Wrong PMI data type"
End If
```

```
Case 6
  Debug.Print "Annotation: Note"

Case 7
  Debug.Print "Annotation: SF Symbol"

Case 8
  Debug.Print "Annotation: Weld Symbol"

Case 9
  Debug.Print "Annotation: Custom Symbol"

Case 10
  Debug.Print "Annotation: Dowel Symbol"

Case 11
  Debug.Print "Annotation: Leader"

Case 12
  Debug.Print "Annotation: Block"

Case 13
  Debug.Print "Annotation: Centermark symbol"

Case 14
  Debug.Print "Annotation: Table"

Case 15
  Debug.Print "Annotation: Centerline"

Case 16
  Debug.Print "Annotation: Datum Origin"

End Select
Next

End If
End Sub
```

Get Points of Repeating Elements in Table-driven Pattern (C#)

This example shows that the points that describe the locations of the repeating elements in a table-driven pattern are based on the table-driven pattern's coordinate system.

```
//-----
// Preconditions:
// 1. Verify that the part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Gets the coordinates of the
//    points for the two repeating elements in the
//    table-driven pattern.
// 2. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere, do
// not save changes.
//-----

using SolidWorks.Interop.sldworks;

using SolidWorks.Interop.swconst;

using System;

using System.Diagnostics;

namespace PatternCS.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);

            ModelDocExtension swModelDocExt = default(ModelDocExtension);

            SelectionMgr swSelMgr = default(SelectionMgr);

            Feature swFeat = default(Feature);

            CoordinateSystemFeatureData swCoordinateData = default(CoordinateSystemFeatureData);

            TablePatternFeatureData swTablePatternFeatData = default(TablePatternFeatureData);

            MathTransform swMathTransform = default(MathTransform);

            MathUtility swMathUtility = default(MathUtility);

            MathPoint swMathPoint = default(MathPoint);

            bool status = false;

            int errors = 0;

            int warnings = 0;

            string filename = null;

            double[]
points = null;

            string point = null;

            double[] pointsArray = new double[3];

            int i = 0;

            filename = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\tablepattern.sldprt";

            swModel = (ModelDoc2)swApp.OpenDoc6(filename, (int)swDocumentTypes_e.swDocPART,
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            swModelDocExt = (ModelDocExtension)swModel.Extension;

            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            // Select the table-driven
pattern

            status = swModelDocExt.SelectByID2("TPattern1", "BODYFEATURE",
```

```

0, 0, 0, false,
0, null,
0);

swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);

swTablePatternFeatData = (TablePatternFeatureData)swFeat.GetDefinition();
swTablePatternFeatData.AccessSelections(swModel, null);

// Get the points of the
repeating elements in the table-driven pattern
//
and transform them to coordinates

swCoordinateData = ((Feature)swTablePatternFeatData.CoordinateSystem).GetDefinition() as CoordinateSystemFeatureData;
swCoordinateData.AccessSelections(swModel, null);
swMathTransform = swCoordinateData.Transform;
swCoordinateData.ReleaseSelectionAccess();

swMathUtility = (MathUtility)swApp.GetMathUtility();

points = (double[])swTablePatternFeatData.PointArray;
Debug.Print("Number of points: " + swTablePatternFeatData.GetPointCount());
for (i = 0; i < points.Length; i += 3)
{
    pointsArray[0] = points[i];
    pointsArray[1] = points[i + 1];
    pointsArray[2] = points[i + 2];
    swMathPoint = (MathPoint)swMathUtility.CreatePoint(pointsArray);

    swMathPoint = swMathPoint.MultiplyTransform(swMathTransform.Inverse()) as MathPoint;

    // Print the coordinates for the two repeating elements in the table-driven pattern
    point = "x: " + ((double[])swMathPoint.ArrayData)[0].ToString() + " y: " + ((double[])swMathPoint.ArrayData)
[1].ToString() + " z: " + ((double[])swMathPoint.ArrayData)[2].ToString();

    Debug.Print(point);
}

swTablePatternFeatData.ReleaseSelectionAccess();
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Get Points of Repeating Elements in Table-driven Pattern (VBA)

This example shows that the points that describe the locations of the repeating elements in a table-driven pattern are based on the table-driven pattern's coordinate system.

```

'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the coordinates of the
'    points for the two repeating elements in the
'    table-driven pattern.
' 2. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeat As SldWorks.Feature
Dim swCoordinateData As SldWorks.CoordinateSystemFeatureData
Dim swTablePatternFeatData As SldWorks.TablePatternFeatureData
Dim swMathTransform As SldWorks.MathTransform
Dim swMathUtility As SldWorks.MathUtility
Dim swMathPoint As SldWorks.MathPoint
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim filename As String
Dim points As Variant
Dim point As String
Dim pointsArray(2) As Double
Dim i As Integer

Sub main()

    filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\tablepattern.sldprt"
    Set swApp = Application.SldWorks

    Set swModel = swApp.OpenDoc6(filename, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    ' Select the table-driven pattern
    status = swModelDocExt.SelectByID2("TPattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)

    Set swTablePatternFeatData = swFeat.GetDefinition
    swTablePatternFeatData.AccessSelections swModel, Nothing

    ' Get the points of the repeating elements in a table-driven pattern
    ' and transform them to coordinates
    Set swCoordinateData = swTablePatternFeatData.CoordinateSystem.GetDefinition
    swCoordinateData.AccessSelections swModel, Nothing
    Set swMathTransform = swCoordinateData.Transform
    swCoordinateData.ReleaseSelectionAccess

    Set swMathUtility = swApp.GetMathUtility

    points = swTablePatternFeatData.PointArray

    Debug.Print "Number of points: " & swTablePatternFeatData.GetPointCount

    For i = 0 To UBound(points) Step 3
        pointsArray(0) = points(i): pointsArray(1) = points(i + 1): pointsArray(2) = points(i + 2)
        Set swMathPoint = swMathUtility.CreatePoint(pointsArray)

        Set swMathPoint = swMathPoint.MultiplyTransform(swMathTransform.Inverse)

        ' Print the coordinates for the two repeating elements in the table-driven pattern
        point = "x: " & swMathPoint.ArrayData(0) & " y: " & swMathPoint.ArrayData(1) & " z: " & swMathPoint.ArrayData(2)

        Debug.Print point
    Next

    swTablePatternFeatData.ReleaseSelectionAccess

End Sub

```

Get Points of Repeating Elements in Table-driven Pattern (VB.NET)

This example shows that the points that describe the locations of the repeating elements in a table-driven pattern are based on the table-driven pattern's coordinate system.

```

'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the coordinates of the
'    points for the two repeating elements in the
'    table-driven pattern.
' 2. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do
' not save changes.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim swCoordinateData As CoordinateSystemFeatureData
        Dim swTablePatternFeatData As TablePatternFeatureData
        Dim swMathTransform As MathTransform
        Dim swMathUtility As MathUtility
        Dim swMathPoint As MathPoint
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim filename As String
        Dim points As Object
        Dim point As String
        Dim pointsArray(2) As Double
        Dim i As Integer

        filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\tablepattern.sldprt"

        swModel = swApp.OpenDoc6(filename, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "",
        errors, warnings)

        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager

        ' Select the table-driven
        pattern
        status = swModelDocExt.SelectByID2("TPattern1", "BODYFEATURE",
        0, 0, 0, False,
        0, Nothing,
        0)

        swFeat = swModel.SelectionManager.GetSelectedObject6(1, -1)

        swTablePatternFeatData = swFeat.GetDefinition
        swTablePatternFeatData.AccessSelections(swModel, Nothing)
    
```

```

        ' Get the points of the repeating
elements in the table-driven pattern
    '
    and transform them to coordinates

        swCoordinateData = swTablePatternFeatData.CoordinateSystem.GetDefinition
        swCoordinateData.AccessSelections(swModel, Nothing)

        swMathTransform = swCoordinateData.Transform

        swCoordinateData.ReleaseSelectionAccess()

        swMathUtility = swApp.GetMathUtility

        points = swTablePatternFeatData.PointArray
    Debug.Print("Number of points: " & swTablePatternFeatData.GetPointCount)
    For i = 0 To UBound(points) Step 3
        pointsArray(0) = points(i) : pointsArray(1) = points(i + 1) : pointsArray(2) = points(i + 2)
        swMathPoint = swMathUtility.CreatePoint(pointsArray)

        swMathPoint = swMathPoint.MultiplyTransform(swMathTransform.Inverse)

        ' Print the coordinates for the two repeating elements in the table-driven pattern
        point = "x: " & swMathPoint.ArrayData(0) & " y: " & swMathPoint.ArrayData(1) & " z: " & swMathPoint.ArrayData(2)

        Debug.Print(point)

    Next

    swTablePatternFeatData.ReleaseSelectionAccess()
End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Get Polyline Data from Drawing View Example (VBA)

This example shows how to retrieve the defining data of arcs, circles, ellipses, splines, and lines in a drawing view.

```
' -----
' Preconditions: Open a drawing of a part.
'
' Postconditions: Inspect the Immediate window for edge and geometry data.
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
```

```
Sub main()
```

```
    Dim swModel                As SldWorks.ModelDoc2
    Dim swDrawing              As SldWorks.DrawingDoc
    Dim swView                 As SldWorks.View
    Dim swSheetView            As SldWorks.View
    Dim swSketch                As SldWorks.Sketch
```

```
    ' Get SOLIDWORKS Design application.
    Set swApp = Application.SldWorks

    ' Get active document.
    Set swModel = swApp.ActiveDoc
```

```
    ' Downcast model to a drawing.
    Set swDrawing = swModel

    ' The first view is the drawing sheet.
    Set swSheetView = swDrawing.GetFirstView
```

```
    ' Print its contents.
    PrintView swSheetView
```

```
    ' Get the sketch for the drawing sheet view.
    Set swSketch = swSheetView.GetSketch

    ' Print its contents.
    PrintSketch swSketch
```

```
    ' Traverse all "real" views on the sheet.
    '
    ' First view on the sheet.
    Set swView = swSheetView.GetNextView
```

```
Do While Not swView Is Nothing
```

```
    PrintView swView
```

```
    Set swSketch = swView.GetSketch
```

```
    PrintSketch swSketch
```

```
    ' Go to next view on the sheet.
    Set swView = swView.GetNextView
```

```
Loop
```

```
End Sub
```

```
Function PrintView(swView As SldWorks.View) As Boolean
```

```
    Dim vEdges As Variant
    Dim vPolyLinesBuffer As Variant
    Dim vLines As Variant
    Dim lNumGeomData As Long
    Dim i As Integer
    Dim iGeomIndex As Integer
    Dim lNumLines As Long
    Dim lItemType As Long
    Dim lBufferSize As Long
    Dim lBufferIdx As Long
    Dim lGeomDataSize As Long
    Dim dGeomData(11) As Double
    Dim lLineData(3) As Long
    Dim lLayerData(1) As Long
    Dim lNumPoints As Long
    Dim dPoint(2) As Double
    Dim lGeomDataIdx As Long
    Dim lLineDataIdx As Long
    Dim lLayerDataIdx As Long
```

```

Dim lStartIdx          As Long
Dim lEndIdx            As Long
Dim lLineDataSize      As Long
Dim lLayerDataSize     As Long
Dim lNumProjectedElements As Long
Dim lNumSketchedElements As Long

If swView Is Nothing Then
    Exit Function
End If

Debug.Print "+++++"
Debug.Print "View = " + swView.Name

'
' Report projected model geometry.
'
' Initialize the number of projected elements for data is contained in the buffer.
lNumProjectedElements = 0

' Get the edges and put all the polyline data into a buffer.
' - no cross-hatch lines.
vEdges = swView.GetPolyLines7(1, vPolyLinesBuffer)

If Not IsEmpty(vEdges) Then
    Debug.Print "Number of edges: " & (UBound(vEdges) - LBound(vEdges) + 1)
End If

' Any polyline data present ?
If Not IsEmpty(vPolyLinesBuffer) Then

    ' Entries for line data and layer data have a fixed size.
    lLineDataSize = 4
    lLayerDataSize = 2

    ' Get the total buffer size.
    lBufferSize = UBound(vPolyLinesBuffer) - LBound(vPolyLinesBuffer) + 1

    ' We start to traverse the buffer at index 0.
    lBufferIdx = 0

    ' Traverse the buffer, consuming data elements.
    Do While lBufferIdx < lBufferSize

        ' Determine type.
        lItemType = vPolyLinesBuffer(lBufferIdx)
        lBufferIdx = lBufferIdx + 1

        ' We have found another projected element.
        lNumProjectedElements = lNumProjectedElements + 1

        ' Handle type specific data.
        If lItemType = 0 Then

            ' HERE: polylines

            Debug.Print "    polyline"

            ' Get GeomDataSize:
            ' - should be zero, but consume it anyway.
            lGeomDataSize = vPolyLinesBuffer(lBufferIdx)
            lBufferIdx = lBufferIdx + 1

            ' Ignore GeomData.

        Else

            ' HERE: arc:
            ' - next to the piecewise linear approximation of the projected model geometry,
            '   the buffer also contains the arc definining the model geometry projection.

            Debug.Print "    arc"

            ' Get GeomDataSize:
            ' - should be 12.
            lGeomDataSize = vPolyLinesBuffer(lBufferIdx)
            lBufferIdx = lBufferIdx + 1

            ' Get GeomData.
            lGeomDataIdx = 0
            lStartIdx = lBufferIdx
            lEndIdx = lStartIdx + (lGeomDataSize - 1)
            For lBufferIdx = lStartIdx To lEndIdx

```

```

        dGeomData(lGeomDataIdx) = vPolyLinesBuffer(lBufferIdx)

        lGeomDataIdx = lGeomDataIdx + 1

    Next lBufferIdx

    Debug.Print "      center pt = (" & dGeomData(0) * 1000# & ", " & dGeomData(1) * 1000# & ", " & dGeomData(2) *
1000# & ") mm"
    Debug.Print "      start  pt = (" & dGeomData(3) * 1000# & ", " & dGeomData(4) * 1000# & ", " & dGeomData(5) *
1000# & ") mm"
    Debug.Print "      end    pt = (" & dGeomData(6) * 1000# & ", " & dGeomData(7) * 1000# & ", " & dGeomData(8) *
1000# & ") mm"
    Debug.Print "      normal   = (" & dGeomData(9) & ", " & dGeomData(10) & ", " & dGeomData(11) & ")"

End If

' Get line data.
lLineDataIdx = 0
lStartIdx = lBufferIdx
lEndIdx = lStartIdx + (lLineDataSize - 1)

For lBufferIdx = lStartIdx To lEndIdx

    lLineData(lLineDataIdx) = vPolyLinesBuffer(lBufferIdx)

    lLineDataIdx = lLineDataIdx + 1

Next lBufferIdx

' Get layer data.
lLayerDataIdx = 0
lStartIdx = lBufferIdx
lEndIdx = lStartIdx + (lLayerDataSize - 1)

For lBufferIdx = lStartIdx To lEndIdx

    lLayerData(lLayerDataIdx) = vPolyLinesBuffer(lBufferIdx)

    lLayerDataIdx = lLayerDataIdx + 1

Next lBufferIdx

' Get point data.
lNumPoints = vPolyLinesBuffer(lBufferIdx)

Debug.Print "      #points = " & CStr(lNumPoints)

lBufferIdx = lBufferIdx + 1
lStartIdx = lBufferIdx
lEndIdx = lStartIdx + lNumPoints * 3 - 1

For lBufferIdx = lStartIdx To lEndIdx Step 3

    dPoint(0) = vPolyLinesBuffer(lBufferIdx)
    dPoint(1) = vPolyLinesBuffer(lBufferIdx + 1)
    dPoint(2) = vPolyLinesBuffer(lBufferIdx + 2)

Next lBufferIdx

Loop
End If

' Report the number of projected elements we found.
Debug.Print " #projected elements = " & CStr(lNumProjectedElements)

'
' Report sketched geometry; only show lines.
'

' Initialize to zero.
lNumSketchedElements = 0

lNumLines = swView.GetLineCount2(1)
If lNumLines <> 0 Then
    vLines = swView.GetLines4(1)

```

```

If Not IsEmpty(vLines) Then

    For i = 0 To lNumLines - 1

        Debug.Print "    line[" & i & "]"

        Debug.Print "        start pt = (" & vLines(i * 12 + 6) * 1000# & ", " & vLines(i * 12 + 7) * 1000# & ", " & vLines(i
* 12 + 8) * 1000# & ") mm"
        Debug.Print "        end   pt = (" & vLines(i * 12 + 9) * 1000# & ", " & vLines(i * 12 + 10) * 1000# & ", " &
vLines(i * 12 + 11) * 1000# & ") mm"

        Next i

    End If

    lNumSketchedElements = lNumSketchedElements + lNumLines
    lNumSketchedElements = lNumSketchedElements + swView.GetArcCount
    lNumSketchedElements = lNumSketchedElements + swView.GetEllipseCount
    lNumSketchedElements = lNumSketchedElements + swView.GetParabolaCount

    ' Report the number of sketched elements found.
    Debug.Print " #sketched elements = " & CStr(lNumSketchedElements)
End Function

Function PrintSketch(swSketch As SldWorks.Sketch) As Boolean

    Dim vSegments          As Variant
    Dim lNumSegments       As Long

    If swSketch Is Nothing Then
        Debug.Print "No Sketch"
    End If

    ' Get the sketch segments.
    vSegments = swSketch.GetSketchSegments

    ' Determine number of segments.
    If Not IsEmpty(vSegments) Then
        lNumSegments = UBound(vSegments) - LBound(vSegments) + 1
    Else
        lNumSegments = 0
    End If

    Debug.Print "Sketch = "
    Debug.Print " #points  = " & CStr(swSketch.GetUserPointsCount())
    Debug.Print " #segments = " & CStr(lNumSegments)

    ' Get count of specific sketch segments.
    If lNumSegments > 0 Then

        Debug.Print "    #arcs      = " & CStr(swSketch.GetArcCount())
        Debug.Print "    #lines     = " & CStr(swSketch.GetLineCount2(1))
        Debug.Print "    #ellipses = " & CStr(swSketch.GetEllipseCount())
        Debug.Print "    #parabolas = " & CStr(swSketch.GetParabolaCount())

    End If

End Function

```

Get Polylines Information Example (VBA)

This example shows how to use IView::GetPolyLines5 and IView::GetSketch. This example also illustrates the differences between projected model geometry and sketched geometry.

```
'-----
' Preconditions:
' 1. Open a drawing that contains:
'   * drawing views of a model comprised of arcs, circles,
'     ellipses, splines, and straight lines.
'   * sketched arcs, circles, ellipses, splines, and
'     straight lines on the drawing.
' 2. Verify that c:\temp exists.
'
' Postconditions:
' 1. Gets the drawing's polylines and any sketches on the
'   drawing information.
' 2. Open c:\temp\PolyLinesInformation.txt in a text editor
'   and examine the contents of the file.
'-----
```

Option Explicit

Dim swApp As SldWorks.SldWorks

Sub main()

```
    Dim swModel As SldWorks.ModelDoc2
    Dim swDrawing As SldWorks.DrawingDoc
    Dim swView As SldWorks.View
    Dim swSheetView As SldWorks.View
    Dim swSketch As SldWorks.Sketch

    ' Get SOLIDWORKS Design application
    Set swApp = Application.SldWorks

    ' Get active document
    Set swModel = swApp.ActiveDoc

    ' Open output file for polylines information
    Open "c:\temp\PolyLinesInformation.txt" For Output As #1

    ' Downcast model to a drawing
    Set swDrawing = swModel

    ' Get the first view, which is the drawing sheet
    Set swSheetView = swDrawing.GetFirstView

    ' Print its contents
    PrintView swSheetView

    ' Get the sketch for the drawing sheet view
    Set swSketch = swSheetView.GetSketch

    ' Print its contents
    PrintSketch swSketch

    ' Traverse all drawing views on the sheet
    ' First drawing view on the sheet
    Set swView = swSheetView.GetNextView
    Do While Not swView Is Nothing
        PrintView swView
        Set swSketch = swView.GetSketch
        PrintSketch swSketch
        ' Go to next drawing view on the sheet
        Set swView = swView.GetNextView
    Loop

    ' Close c:\temp\PolyLinesInformation.txt
    Close #1
```

End Sub

Function PrintView(swView As SldWorks.View) As Boolean

```
    Dim vPolyLinesBuffer As Variant
    Dim vLines As Variant
    Dim lNumGeomData As Long
    Dim i As Integer
    Dim iGeomIndex As Integer
    Dim lNumLines As Long
    Dim lItemType As Long
    Dim lBufferSize As Long
    Dim lBufferIdx As Long
    Dim lGeomDataSize As Long
    Dim dGeomData(11) As Double
    Dim lLineData(3) As Long
    Dim lLayerData(1) As Long
    Dim lNumPoints As Long
    Dim dPoint(2) As Double
    Dim lGeomDataIdx As Long
    Dim lLineDataIdx As Long
    Dim lLayerDataIdx As Long
    Dim lStartIdx As Long
    Dim lEndIdx As Long
    Dim lLineDataSize As Long
    Dim lLayerDataSize As Long
    Dim lNumProjectedElements As Long
    Dim lNumSketchedElements As Long
```

```

If swView Is Nothing Then
    Exit Function
End If

Write #1, ""
Write #1, "View = " + swView.GetName2

' Report projected model geometry
' Initialize the number of projected elements for data is contained in the buffer
lNumProjectedElements = 0

' Get all the polyline data into a buffer
' No cross-hatch lines
vPolyLinesBuffer = swView.GetPolyLines5(0)

' Any polyline data present ?
If Not IsEmpty(vPolyLinesBuffer) Then
    ' Entries for line data and layer data have a fixed size
    lLineDataSize = 4
    lLayerDataSize = 2
    ' Get the total buffer size
    lBufferSize = UBound(vPolyLinesBuffer) - LBound(vPolyLinesBuffer) + 1
    ' Start to traverse the buffer at index 0
    lBufferIdx = 0
    ' Traverse the buffer, consuming data elements
    Do While lBufferIdx < lBufferSize
        ' Determine type
        lItemType = vPolyLinesBuffer(lBufferIdx)
        lBufferIdx = lBufferIdx + 1
        ' Found another projected element
        lNumProjectedElements = lNumProjectedElements + 1
        ' Handle type specific data
        If lItemType = 0 Then
            ' HERE: polyline
            Write #1, "    Polyline"
            ' Get GeomDataSize:
            ' should be zero, but consume it anyway
            lGeomDataSize = vPolyLinesBuffer(lBufferIdx)
            lBufferIdx = lBufferIdx + 1
            ' Ignore GeomData
        Else
            ' HERE: arc:
            ' Next to the piecewise linear approximation of the projected model geometry,
            ' the buffer also contains the arc defining the model geometry projection
            Write #1, "    Arc"
            ' Get GeomDataSize:
            ' should be 12
            lGeomDataSize = vPolyLinesBuffer(lBufferIdx)
            lBufferIdx = lBufferIdx + 1
            ' Get GeomData
            lGeomDataIdx = 0
            lStartIdx = lBufferIdx
            lEndIdx = lStartIdx + (lGeomDataSize - 1)
            For lBufferIdx = lStartIdx To lEndIdx
                dGeomData(lGeomDataIdx) = vPolyLinesBuffer(lBufferIdx)
                lGeomDataIdx = lGeomDataIdx + 1
            Next lBufferIdx
            Write #1, "        Center point = (" & dGeomData(0) * 1000# & ", " & dGeomData(1) * 1000# & ", " & dGeomData(2) * 1000# & ") mm"
            Write #1, "        Start point = (" & dGeomData(3) * 1000# & ", " & dGeomData(4) * 1000# & ", " & dGeomData(5) * 1000# & ") mm"
            Write #1, "        End point = (" & dGeomData(6) * 1000# & ", " & dGeomData(7) * 1000# & ", " & dGeomData(8) * 1000# & ") mm"
            Write #1, "        Normal = (" & dGeomData(9) & ", " & dGeomData(10) & ", " & dGeomData(11) & ") "
        End If

        ' Get line data
        lLineDataIdx = 0
        lStartIdx = lBufferIdx
        lEndIdx = lStartIdx + (lLineDataSize - 1)
        For lBufferIdx = lStartIdx To lEndIdx
            lLineData(lLineDataIdx) = vPolyLinesBuffer(lBufferIdx)
            lLineDataIdx = lLineDataIdx + 1
        Next lBufferIdx
        ' Get layer data
        lLayerDataIdx = 0
        lStartIdx = lBufferIdx
        lEndIdx = lStartIdx + (lLayerDataSize - 1)
        For lBufferIdx = lStartIdx To lEndIdx
            lLayerData(lLayerDataIdx) = vPolyLinesBuffer(lBufferIdx)
            lLayerDataIdx = lLayerDataIdx + 1
        Next lBufferIdx
        ' Get point data
        If lBufferIdx <= UBound(vPolyLinesBuffer) Then
            lNumPoints = vPolyLinesBuffer(lBufferIdx)
            Write #1, "        Number of points = " & CStr(lNumPoints)
            lBufferIdx = lBufferIdx + 1
            lStartIdx = lBufferIdx
            lEndIdx = lStartIdx + lNumPoints * 3 - 1
            For lBufferIdx = lStartIdx To lEndIdx Step 3
                dPoint(0) = vPolyLinesBuffer(lBufferIdx)
                dPoint(1) = vPolyLinesBuffer(lBufferIdx + 1)
                dPoint(2) = vPolyLinesBuffer(lBufferIdx + 2)
            Next lBufferIdx
        End If
    Loop
End If

' Report the number of projected elements found
Write #1, "    Number of projected elements = " & CStr(lNumProjectedElements)

' Report sketched geometry:
' only show lines

' Initialize to zero
lNumSketchedElements = 0
lNumLines = swView.GetLineCount2(1)

```

```

If lNumLines <> 0 Then
    vLines = swView.GetLines4(1)
    If Not IsEmpty(vLines) Then
        For i = 0 To lNumLines - 1
            Write #1, "    line[" & i & "]"
            Write #1, "        Start point = (" & vLines(i * 12 + 6) * 1000# & ", " & vLines(i * 12 + 7) * 1000# & ", " & vLines(i * 12 + 8) * 1000# &
") mm"
            Write #1, "        End    point = (" & vLines(i * 12 + 9) * 1000# & ", " & vLines(i * 12 + 10) * 1000# & ", " & vLines(i * 12 + 11) * 1000# &
") mm"
        Next i
    End If
End If
lNumSketchedElements = lNumSketchedElements + lNumLines
lNumSketchedElements = lNumSketchedElements + swView.GetArcCount
lNumSketchedElements = lNumSketchedElements + swView.GetEllipseCount
lNumSketchedElements = lNumSketchedElements + swView.GetParabolaCount

' Report the number of sketched elements found
Write #1, "    Number of sketched elements = " & CStr(lNumSketchedElements)

End Function

Function PrintSketch(swSketch As SldWorks.Sketch) As Boolean
    Dim vSegments As Variant
    Dim lNumSegments As Long

    If swSketch Is Nothing Then
        Write #1, "No sketch"
    End If

    ' Get the sketch segments
    vSegments = swSketch.GetSketchSegments

    ' Determine number of segments
    If Not IsEmpty(vSegments) Then
        lNumSegments = UBound(vSegments) - LBound(vSegments) + 1
    Else
        lNumSegments = 0
    End If

    Write #1, "Sketch = "
    Write #1, "    Number of points  = " & CStr(swSketch.GetUserPointsCount())
    Write #1, "    Number of segments = " & CStr(lNumSegments)

    ' Get number of specific sketch segments
    If lNumSegments > 0 Then
        Write #1, "        Number of arcs      = " & CStr(swSketch.GetArcCount())
        Write #1, "        Number of lines     = " & CStr(swSketch.GetLineCount2(1))
        Write #1, "        Number of ellipses  = " & CStr(swSketch.GetEllipseCount())
        Write #1, "        Number of parabolas = " & CStr(swSketch.GetParabolaCount())
    End If

End Function

```

Get Precisions for a Dimension Example (VBA)

This example shows how to get the precisions for a dimension.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\lesson2\tutor1.sldprt.  
' 2. Right-click Annotations and click Show Feature Dimensions.  
' 3. Open the Immediate window.  
'  
' Postconditions: Examine the Immediate window.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
'-----  
Option Explicit  
  
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swDispDim As SldWorks.DisplayDimension  
    Dim swDim As SldWorks.Dimension  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    bRet = swModel.Extension.SelectByID2("D1@Fillet1@tutor1.SLDPRT", "DIMENSION", 1.2727536990925E-02, 2.29950387838887E-02, 7.75856501351018E-02, False,  
0, Nothing, 0)  
    Set swDispDim = swSelMgr.GetSelectedObject6(1, -1)  
    Set swDim = swDispDim.GetDimension  
    Debug.Print "Dim = " & swDim.FullName  
    Debug.Print "    Nominal:"  
    Debug.Print "        Primary      = " & swDispDim.GetPrimaryPrecision2  
    Debug.Print "        Alternate    = " & swDispDim.GetAlternatePrecision2  
    Debug.Print "    Tolerance:"  
    Debug.Print "        Primary      = " & swDispDim.GetPrimaryTolPrecision2  
    Debug.Print "        Alternate    = " & swDispDim.GetAlternateTolPrecision2  
End Sub
```

Get Preselected Object Example (C#)

This example shows how to get a preselected object when a preselection event is fired.

```
//-----
// Preconditions:
// 1. Open a part, assembly, or drawing document.
// 2. Click Tools > Options > System Options > Stop VSTA debugger
//    on macro exit if the check box is selected.
// 3. If a part or assembly document is active, then
//    preselect (mouse over) a face.
//    - or -
//    If a drawing document is active, then pre-select
//    an edge.
// 4. Open the Immediate window.
//
// Postconditions:
// 1. When a face is preselected in a part or assembly
//    document or an edge is pre-selected in a drawing document,
//    then that interface's UserSelectionPreNotify event is fired.
// 2. To stop running the macro, click Debug > Stop Debugging
//    in the SolidWorks Visual Studio Tools for Applications IDE.
// 3. Examine the Immediate window.
// 4. Click Tools > Options > System Options > Stop VSTA debugger
//    on macro exit to select the check box.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Collections;
using System.Diagnostics;
namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public PartDoc pDoc;
        public AssemblyDoc aDoc;
        public DrawingDoc dDoc;
        public void Main()
        {
            ModelDoc2 swModel;
            Hashtable openPart;
            Hashtable openAssembly;
            Hashtable openDrawing;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Determine the document type
            // and set up event handlers
            if (swModel.GetType() == (int)swDocumentTypes_e.swDocPART)
            {
                pDoc = (PartDoc)swModel;
                openPart = new Hashtable();
            }
            else if (swModel.GetType() == (int)swDocumentTypes_e.swDocASSEMBLY)
            {
                aDoc = (AssemblyDoc)swModel;
                openAssembly = new Hashtable();
            }
            else if (swModel.GetType() == (int)swDocumentTypes_e.swDocDRAWING)
            {
                dDoc = (DrawingDoc)swModel;
                openDrawing = new Hashtable();
            }
            AttachEventHandlers();
        }
        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }
        public void AttachSWEvents()
        {
            if ((pDoc != null))
            {
                pDoc.UserSelectionPreNotify += this.pDoc_UserSelectionPreNotify;
            }
            if ((aDoc != null))
            {
                aDoc.UserSelectionPreNotify += this.aDoc_UserSelectionPreNotify;
            }
            if ((dDoc != null))
            {
                dDoc.UserSelectionPreNotify += this.dDoc_UserSelectionPreNotify;
            }
        }
        private int pDoc_UserSelectionPreNotify(int SelectionType)
        {
            int functionReturnValue = 0;
            ModelDoc2 swModel;
            SelectionMgr swSelMgr;
            object SelectedObject;
            Face2 swFace;
            Feature swFeature;
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            SelectedObject = (object)swSelMgr.GetPreSelectedObject();
            if (SelectionType == (int)swSelectType_e.swSelFACES)
            {
                swFace = (Face2)SelectedObject;
                swFeature = (Feature)swFace.GetFeature();
                Debug.Print("Name of feature whose face you preselected: " + swFeature.Name);
                Debug.Print(" Mouse over a different face, or click Debug > Stop Debugging to stop running macro");
                Debug.Print(" ");
                functionReturnValue = 1;
            }
            return functionReturnValue;
        }
        public int aDoc_UserSelectionPreNotify(int SelectionType)
        {
        }
    }
}
```

```

    int functionReturnValue = 0;
    ModelDoc2 swModel;
    SelectionMgr swSelMgr;
    object SelectedObject;
    Face2 swFace;
    Feature swFeature;
    swModel = (ModelDoc2)swApp.ActiveDoc;
    swSelMgr = (SelectionMgr)swModel.SelectionManager;
    SelectedObject = (object)swSelMgr.GetPreSelectedObject();
    if (SelectionType == (int)swSelectType_e.swSelFACES)
    {
        swFace = (Face2)SelectedObject;
        swFeature = (Feature)swFace.GetFeature();
        Debug.Print("Name of feature whose face you preselected: " + swFeature.Name);
        Debug.Print("    Mouse over a different face, or click Debug > Stop Debugging to stop running macro");
        Debug.Print(" ");
        functionReturnValue = 1;
    }
    return functionReturnValue;
}
private int dDoc_UserSelectionPreNotify(int SelectionType)
{
    int functionReturnValue = 0;
    ModelDoc2 swModel;
    SelectionMgr swSelMgr;
    object SelectedObject;
    Edge swEdge;
    Body2 swBody;
    swModel = (ModelDoc2)swApp.ActiveDoc;
    swSelMgr = (SelectionMgr)swModel.SelectionManager;
    SelectedObject = (object)swSelMgr.GetPreSelectedObject();
    if (SelectionType == (int)swSelectType_e.swSelEDGES)
    {
        swEdge = (Edge)SelectedObject;
        swBody = (Body2)swEdge.GetBody();
        Debug.Print("Name of body whose edge you preselected: " + swBody.Name);
        Debug.Print("    Mouse over a different edge, or click Debug > Stop Debugging to stop running macro");
        Debug.Print(" ");
        functionReturnValue = 1;
    }
    return functionReturnValue;
}
/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Get Preselected Object Example (VBA)

This example shows how to get a preselected object when a preselection event is fired.

```
'-----
' Preconditions:
' 1. Open a part, assembly, or drawing.
' 2. In the SOLIDWORKS Visual Basic IDE:
'   a. Copy and paste Main into the macro's main module.
'   b. Click Insert > Class Module and copy and paste
'      Class1 into that module.
'   c. Click Insert > Class Module and copy and paste
'      Class2 into that module.
'   d. Click Insert > Class Module and copy and paste
'      Class3 into that module.
' 3. If a part or assembly document is active, then
'    preselect (mouse over) a face.
'    - or -
'    If a drawing document is active, then preselect
'    an edge.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. When a face is preselected in a part or assembly
'    document, or an edge is preselected in a drawing document,
'    then that interface's UserSelectionPreNotify event is fired.
' 2. In the SOLIDWORKS Visual Basic IDE, click Run > Reset to
'    stop running the macro.
' 3. Examine the Immediate window.
'-----
' Main
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim partDoc As New Class1
Dim assemblyDoc As New Class2
Dim drawingDoc As New Class3

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    ' Determine the document type, then
    ' execute its corresponding class module
    If swModel.GetType = swDocPART Then
        partDoc.init swModel
    ElseIf swModel.GetType = swDocASSEMBLY Then
        assemblyDoc.init swModel
    ElseIf swModel.GetType = swDocDRAWING Then
        drawingDoc.init swModel
    End If

End Sub

Go to top

'Class1

Public WithEvents doc As partDoc

Public Function init(ByRef docIn As Object)
    Set doc = docIn
End Function

Private Function doc_UserSelectionPreNotify(ByVal SelectionType As Long) As Long
    If SelectionType = swSelFACES Then
        Set swApp = Application.SldWorks
        Set swModel = swApp.ActiveDoc
        Dim swSelMgr As SldWorks.SelectionMgr
        Dim SelectedObject As Object
        Dim swFace As SldWorks.Face2
        Dim swFeature As SldWorks.Feature
        Set swSelMgr = swModel.SelectionManager
        Set SelectedObject = swSelMgr.GetPreSelectedObject
        If SelectionType = swSelFACES Then
            Set swFace = SelectedObject
            Set swFeature = swFace.GetFeature
            Debug.Print "Name of feature whose face you preselected: " & swFeature.Name
            Debug.Print "    Select a different face, or in the IDE to stop running the macro, click Run > Reset"
            Debug.Print " "
        End If
    End If
End Function

Go to top

'Class2

Public WithEvents doc As assemblyDoc

Public Function init(ByRef docIn As Object)
    Set doc = docIn
End Function
```

```

Private Function doc_UserSelectionPreNotify(ByVal SelectionType As Long) As Long
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim SelectedObject As Object
    Dim swFace As SldWorks.Face2
    Dim swFeature As SldWorks.Feature
    Set swSelMgr = swModel.SelectionManager
    Set SelectedObject = swSelMgr.GetPreSelectedObject
    If SelectionType = swSelFACES Then
        Set swFace = SelectedObject
        Set swFeature = swFace.GetFeature
        Debug.Print "Name of feature whose face you preselected: " & swFeature.Name
        Debug.Print "    Select a different face, or in the IDE to stop running the macro, click Run > Reset"
        Debug.Print " "
    End If
End Function

```

[Go to top](#)

'Class3

Public WithEvents doc As drawingDoc

Public Function init(ByRef docIn As Object)

Set doc = docIn

End Function

```

Private Function doc_UserSelectionPreNotify(ByVal SelectionType As Long) As Long

```

```

    If SelectionType = swSelEDGES Then

```

```

        Set swApp = Application.SldWorks

```

```

        Set swModel = swApp.ActiveDoc

```

```

        Dim swSelMgr As SldWorks.SelectionMgr

```

```

        Dim SelectedObject As Object

```

```

        Dim swEdge As SldWorks.Edge

```

```

        Dim swBody As SldWorks.Body2

```

```

        Set swSelMgr = swModel.SelectionManager

```

```

        Set SelectedObject = swSelMgr.GetPreSelectedObject

```

```

        If SelectionType = swSelEDGES Then

```

```

            Set swEdge = SelectedObject

```

```

            Set swBody = swEdge.GetBody

```

```

            Debug.Print "Name of body whose edge you preselected: " & swBody.Name

```

```

            Debug.Print "    Select a different edge, or in the IDE to stop running the macro, click Run > Reset"

```

```

            Debug.Print " "

```

```

        End If

```

```

    End If

```

```

End Function

```

[Go to top](#)